Grundlagen der Programmierung (Python)

Klausurorientierte Zusammenfassung · DHBW Heilbronn, Wirtschaftsinformatik, 1. Semester · Dozent: Dr. Markus Rosenfelder

Stand: 10.11.2026 · **deckt VL1 bis VL7 ab** (neu: Kap. 6 Vererbung und Polymorphie, Kap. 7 Projekt Kaffeeshop, „for-Schleifen ganz langsam“ in Kap. 3, „print oder return?“ in 4.9, Selbsttest und Spickzettel) und wird nach den nächsten Terminen erweitert (Standardbibliothek, Klausurvorbereitung).

Klausur auf einen Blick

Was bekannt ist

Termin	**Fr 11.12.2026** (laut Lernplan)
Dauer	**120 Minuten**
Form	Klausur, entweder auf **Papier** oder **am Rechner** (noch offen)
Musterklausur	**keine**. Was relevant ist, zeigen die Übungen, die Mini-Aufgaben auf den Folien und die offiziellen Musterlösungen.
Stoffstand	8 von 11 Terminen gehalten (bis 06.11.). VL5 (OOP I) lief am 23.10. und 27.10., VL6 (OOP II) am 03.11. und laut Plan am 06.11., danach begann VL7 (Projekt Kaffeeshop). Ob am 06.11. alles wie geplant drankam, ist nicht bestätigt.
Arbeitsweise im Kurs	Code von Hand im Texteditor, ohne IDE und ohne KI. Python 3.14, Prüfwerkzeuge `ruff` und `ty`

Was noch offen ist

- Papier oder Rechner? Uhrzeit, Raum und erlaubte Hilfsmittel sind noch nicht bekannt.
- Restliche Termine bis 27.11.: Rest Kaffeeshop (Stufe 2/3), VL08 „Standardbibliothek“ (schon in Moodle) und Klausurvorbereitung. **Dieses Dokument wird ergänzt.**
- Ob `typing.Protocol` (Ausblick in VL6) Klausurstoff ist, ist offen. Die Musterlösung zum Kaffeeshop kommt erst nach dem Projekt.
- Ob List Comprehensions und `map` / `filter` / `reduce` prüfungsrelevant sind, ist unklar. Bisher kommen sie nur in einer Musterlösung bzw. als Stichwort vor.

Wahrscheinliche Aufgabentypen **(abgeleitet, nicht bestätigt)**

1. **„Was gibt das aus?“**: Mini-Aufgaben dieser Art stehen direkt auf den Folien (z. B. Modulo, Zuweisungsoperatoren, Vergleiche).
2. **Fehler finden und Fehlermeldungen deuten.** Laut Dozent ist das „die wichtigste Fähigkeit“ (VL1).
3. **Code nach einer Textaufgabe schreiben**, z. B. Staffelpreise, Gebühren, Schleifen über Listen, Funktionen, Rekursion, Klassen und Unterklassen.
4. **Begriffe erklären und vergleichen**, z. B. Compiler oder Interpreter, imperativ oder applikativ, Klasse oder Objekt.

Prioritäten fürs Lernen

1	**Operatoren und Ausdrücke**: `/ // %`, Rangfolge, Typumwandlung (Kap. 2)
2	**Fehlerarten und Tracebacks**: Syntax, Typ, Laufzeit, Logik (Kap. 2, Anhang)
3	**Verzweigungen und Schleifen**: `elif`, `match`, `range`, `break` / `continue` (Kap. 3)
4	**Listen und Namen → Objekte**, Verweis-Falle (Kap. 2–4)
5	**Funktionen**: `return`, Standardwerte, LEGB, Übergabe (Kap. 4)
6	**Rekursion**: Abbruchbedingung, Aufrufstapel (Kap. 4)
7	**Klassen**: `__init__`, `self`, Methoden, Property, `@dataclass` (Kap. 5)
8	**Vererbung und Polymorphie**: `super()`, Überschreiben, abstrakte Klassen, Fähigkeiten (Kap. 6)
9	**Kaffeeshop-Muster**: finden, sammeln, `hinzufuegen` mit Mengenprüfung, Menü (Kap. 7)
10	Theorie VL1: Algorithmus, Übersetzung, Werkzeuge (Kap. 1)

So liest du dieses Dokument

- Klausurrelevant ist, was auf den Folien steht. Was darüber hinausgeht, ist mit *(Ergänzung)* markiert.
- **Orange Kästen** zeigen Folienfehler mit der richtigen Lösung. **Rote Kästen** sammeln typische Fehler und Fallen, **grüne Kästen** sind Checklisten („Das musst du können“).
- Alle Codebeispiele wurden ausgeführt, die Ausgabe ist echt (lokal mit Python 3.13 statt der Kursversion 3.14; einzelne Fehlermeldungstexte können minimal abweichen, z. B. „float division by zero“ statt „division by zero“). Bei Fehlern steht nur die letzte Zeile des Tracebacks da. Aus Platzgründen steht zwischen Funktionen nur eine Leerzeile (`ruff format` setzt zwei).
- Für eine Papierklausur: Achte beim Schreiben auf saubere 4er-Einrückung und auf den Doppelpunkt nach `if` / `for` / `def` / `class`. *(Ergänzung)*

Inhalt

Klausur auf einen Blick 1

VL1 · Einführung: Algorithmen, Übersetzung, Python 3
- 1.1 Algorithmen 3
- 1.2 Datenstrukturen 3
- 1.3 Von der Maschinensprache zu Python 3
- 1.4 Python: Eigenschaften, Syntax, Werkzeuge 4

VL2 · Variablen, Datentypen und Operatoren 5
- 2.1 Kommentare und Docstrings 5
- 2.2 Typisierung und einfache Datentypen 5
- 2.3 Variablen: Namen zeigen auf Werte 5
- 2.4 Operatoren 6
- 2.5 Strings und f-Strings 7
- 2.6 Typumwandlung, Rundung und Geld 7
- 2.7 Syntax vs. Semantik: vier Fehler, drei Prüfer 8

VL3 · Kontrollstrukturen und Listen 9
- 3.1 Drei Grundformen und Blöcke 9
- Kasten: for-Schleifen ganz langsam neu 9
- 3.2 Bedingte Anweisungen: if, elif, else 9
- 3.3 match und der bedingte Ausdruck 10
- 3.4 Wahrheitswerte beliebiger Werte 10
- 3.5 Logische Operatoren, Rangfolge, Kurzschluss 10
- 3.6 Übungen Teil 1: Stromkosten und Parkgebühren 11
- 3.7 Schleifen: while und for 11
- 3.8 break, continue und typische Schleifenfehler 12
- 3.9 Listen 13
- 3.10 Übungen Teil 2 (Musterlösungen) 13

VL4 · Funktionen und Rekursion 15
- 4.1 Funktionen definieren und aufrufen 15
- 4.2 Rückgabewerte und None 15
- 4.3 Gültigkeitsbereiche (LEGB) 16
- 4.4 Parameterübergabe: Namen, keine Kopien 16
- 4.5 Standardwerte, Schlüsselwortargumente, *args statt 17
- 4.6 Imperative vs. applikative Programmierung 18
- 4.7 Rekursion 18
- 4.8 Funktionen testen: assert und pytest 20
- 4.9 print oder return? neu 21

VL5 · Objektorientierte Programmierung I 23
- 5.1 Klassen und Objekte 23
- 5.2 Eine Klasse in Python: __init__ und self 23
- 5.3 Methoden, self, __str__ und __repr__ 24
- 5.4 Namen zeigen auf Objekte, auch bei eigenen Klasser 25
- 5.5 Klassenattribute 25
- 5.6 Kapselung mit Properties 26
- 5.7 Zusammengesetzte Klassen, Module und @dataclas 27

VL6 · OOP II: Vererbung und Polymorphie neu 30
- 6.1 Vererbung: „ist ein“ und super().__init__ 30
- 6.2 Überschreiben, super().methode(), @override und die 31
- 6.3 Polymorphie, Typangabe und Duck Typing 32
- 6.4 object und __repr__ 33
- 6.5 Abstrakte Klassen: ABC und @abstractmethod 33
- 6.6 Fähigkeiten statt Interfaces 34
- 6.7 Wiederholung VL1–VL6: Fitnessstudio 35

VL7 · Projekt Kaffeeshop neu 37
- 7.1 Aufgabe, Ablauf und Startpaket 37
- 7.2 Stufe 1, Teil 1: Das Lager (finden und suchen) 38
- 7.3 Stufe 1, Teil 2: Der Warenkorb und hinzufuegen() 38
- 7.4 Stufe 1, Teil 3: Das Menü in main.py 40
- 7.5 Stufe 2: Produktarten und Zahlungsarten 41
- 7.6 Stufe 3: Zusatzaufgaben nach Wahl 41

Fehlermeldungen und Selbsttest 43
- A.1 Fehlermeldungen auf einen Blick 43
- A.2 Selbsttest: Was gibt das aus? 43

Spickzettel: Python-Syntax auf einer Seite neu 46

VL1 Einführung: Algorithmen, Übersetzung, Python

1.1 Algorithmen

Ein **Algorithmus** ist eine Schrittfolge, die ein Problem systematisch löst. Er beschreibt den Lösungsweg unabhängig von der Programmiersprache. Jeder Schritt ist so genau beschrieben, dass eine Person oder ein Computer ihn ohne Rückfrage ausführen kann. An einen Algorithmus stellt man vier **Anforderungen**: **Endlichkeit** (er endet), **Ausführbarkeit**, **Korrektheit** und **Effizienz** (er geht sparsam mit Zeit und Speicher um).

Eigenschaft	Bedeutung	Merkhilfe
terminierend	endet bei jeder erlaubten Eingabe nach endlich vielen Schritten	Er hört auf.
determiniert	Gleiche Eingaben und Startbedingungen liefern immer das gleiche **Ergebnis**.	Es geht um das Ergebnis.
deterministisch	Der **nächste Schritt** und der Zustand sind zu jedem Zeitpunkt eindeutig festgelegt.	Es geht um den Weg.

Folienbeispiel Nachbestellung: (1) Nimm den nächsten Artikel aus der Lagerliste. (2) Ist der Bestand kleiner als der Meldebestand? Wenn nein, weiter mit 5. (3) Bestellmenge = Höchstbestand − Bestand. (4) Löse die Bestellung aus. (5) Sind noch Artikel in der Liste? Wenn ja, weiter mit 1, sonst fertig. Für die Schrauben (Bestand 12, Meldebestand 20, Höchstbestand 100) werden 100 − 12 = **88 Stück** bestellt, für die Muttern (8 / 25 / 120) **112 Stück**. Bei den Dichtringen (45 / 30) wird nichts bestellt. Gleiche Bestände ergeben immer dieselbe Bestellung (determiniert).

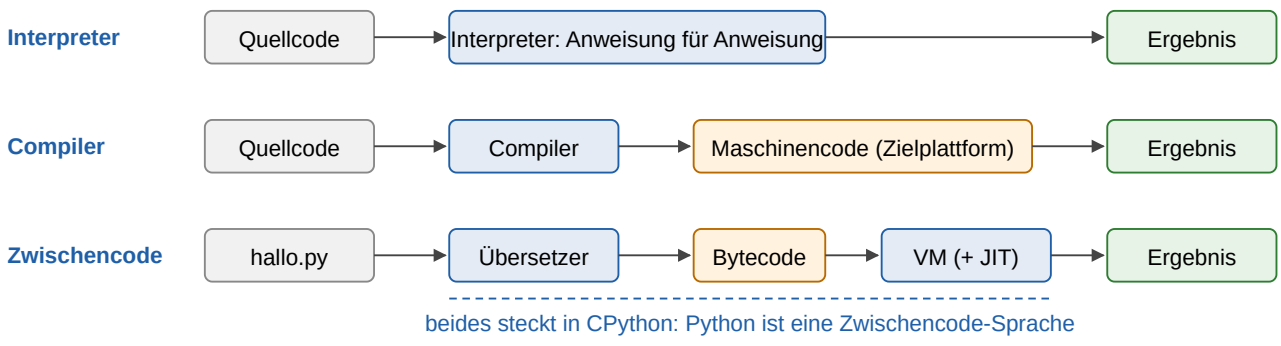
1.2 Datenstrukturen

Datenstrukturen ordnen Daten so, dass man effizient auf sie zugreifen und sie ändern kann. In Python sind `list`, `tuple`, `dict` und `set` fest eingebaut.

Datenstruktur	Eigenschaft	Beispiel
Array	Zugriff über einen Index, feste Größe	Messwerte pro Monat
Liste	flexible Größe	Warenkorb
Baum	Hierarchien	Dateisystem, Ablage nach Jahr → Lieferant → Monat
Graph	Netzwerke	Routenplanung
Stack (Stapel)	**LIFO**: last in, first out	Zurück-Taste im Browser
Queue (Warteschlange)	**FIFO**: first in, first out	Druckaufträge, Rechnungen in Eingangsreihenfolge

1.3 Von der Maschinensprache zu Python

Ein Prozessor versteht nur **Maschinensprache**, also binäre Befehle. **Assembler** schreibt dieselben Befehle symbolisch (`MOV`, `ADD`, `CMP`, `JE`) und hängt deshalb vom Prozessortyp ab. Ab den 1950er-Jahren kamen **höhere Programmiersprachen**, die von der Hardware abstrahieren. Früher teilte man die Sprachen in Generationen ein. Heute spricht man von **Paradigmen**, die nebeneinander bestehen: imperativ bzw. prozedural, objektorientiert, funktional, deklarativ bzw. logisch. Python ist **multiparadigmatisch**, du kannst darin also nach mehreren Paradigmen programmieren.



Drei Wege vom Quellcode zum Ergebnis

Strategie	Ablauf	Vorteil	Nachteil	Beispiele
Interpreter	liest Anweisung für Anweisung und führt sie sofort aus; es entsteht kein dauerhaftes Programm	schnelles Ausprobieren	langsamer	Shell-Skripte

Compiler	übersetzt vorab alles in Maschinencode für die Zielhardware	hohe Geschwindigkeit	vor jedem Test kompilieren; läuft nur auf der Zielplattform	C, C++, Rust, Go
Zwischencode	Bytecode, den eine virtuelle Maschine ausführt (oft mit JIT)	portabel	braucht eine Laufzeitumgebung	Java (JVM), C# (.NET), Python

Ein **JIT** (Just-in-Time-Compiler) übersetzt häufig genutzten Code erst zur Laufzeit in Maschinencode. Dadurch verschwimmen die Grenzen zwischen Interpreter und Compiler. Python hat seit Version 3.13 einen experimentellen JIT.

1.4 Python: Eigenschaften, Syntax, Werkzeuge

Guido van Rossum hat Python ab 1989 entwickelt und 1991 veröffentlicht, den Namen hat es von Monty Python. Python 3 erschien 2008 und ist bewusst inkompatibel zu Python 2, im Kurs wird **Python 3.14** verwendet. Python ist plattformunabhängig, hat eine automatische Speicherverwaltung und eine große Standardbibliothek („Batteries included“). Außerdem ist es **dynamisch, aber stark typisiert**: Jeder *Wert* hat einen festen Typ, Variablen haben keinen. Python wandelt nicht heimlich um, deshalb gibt `"5" + 1` einen Fehler (Kap. 2).

Syntaxregeln

- **Einrückung bildet Blöcke** (Standard: 4 Leerzeichen), geschweifte Klammern gibt es dafür nicht.
- Ein `:` leitet einen Block ein. Ein Semikolon am Zeilenende braucht man nicht.
- Groß- und Kleinschreibung zählt: `Print` ist nicht `print`.
- `#` leitet einen Kommentar ein. Erstes Programm: `print("Hello World!")`
- Eine Klasse oder `main` ist nicht nötig, die Datei läuft von oben nach unten.

Was bei `uv run hallo.py` passiert

1. CPython prüft zuerst die **Syntax** der ganzen Datei. Einen Tippfehler bemerkt es also, bevor eine einzige Zeile läuft.
2. Es übersetzt den Code in **Bytecode**. Für importierte Module wird er in `__pycache__/` gespeichert.
3. Die **Python-VM** führt den Bytecode aus.

Werkzeug	Aufgabe	Befehl
uv	installiert Python, verwaltet Projekte und Pakete, startet Programme	`uv run hallo.py`, `uv add --dev ruff ty`
CPython	Übersetzer und VM in einem, mit Standardbibliothek; Projektumgebung `.venv`	`uv run python` (Konsole, Prompt `>>>`, Ende mit `exit()`)
ruff	Formatierung, Stil und typische Fehler	`uv run ruff format .`, `uv run ruff check .`
ty	prüft die Datentypen, **ohne** das Programm auszuführen	`uv run ty check`

Typische Fehler und Fallen

- „Determiniert“ (gleiches *Ergebnis*) und „deterministisch“ (eindeutiger nächster *Schritt*) werden oft verwechselt.
- Python ist *nicht* „rein interpretiert“. Es übersetzt erst in Bytecode, den dann eine VM ausführt.
- „Dynamisch typisiert“ heißt nicht „schwach typisiert“. Python rät nicht, deshalb gibt `"Alter: " + 36` einen TypeError.

Das musst du können

☐ die vier Anforderungen an Algorithmen nennen und terminierend, determiniert und deterministisch unterscheiden
☐ Stack (LIFO) und Queue (FIFO) erklären und passend zuordnen
☐ Interpreter, Compiler und Zwischencode mit Vor- und Nachteilen vergleichen und Python einordnen
☐ die Syntaxregeln anwenden und sagen, wofür uv, ruff und ty zuständig sind

VL2 Variablen, Datentypen und Operatoren

2.1 Kommentare und Docstrings

Ein **Kommentar** beginnt mit `#` und reicht bis zum Zeilenende. Python ignoriert ihn, er ist für Menschen gedacht. Ein guter Kommentar erklärt das **Warum**, denn das Was zeigt der Code selbst. Für mehrere Zeilen schreibt man mehrere `#`-Zeilen. `/* … */` wie in Java gibt es nicht. Ein **Docstring** `"""…"""` direkt unter `def` oder `class` beschreibt eine Funktion, Klasse oder Datei. Im Kurs gilt der **Google-Stil**: zuerst ein kurzer Satz, dann `Args:` und `Returns:`.

```
# Seit 2007 gilt der Regelsatz von 19 %.   (sinnvoll: erklärt das Warum)
netto = 100
brutto = netto * 1.19  # Multipliziere netto mit 1.19   (überflüssig)

def kreisflaeche(radius: float) -> float:
    """Berechnet die Fläche eines Kreises.

    Args:
        radius: Radius in Metern, nicht negativ.

    Returns:
        Die Fläche in Quadratmetern.
    """
    return 3.14159 * radius**2

print(brutto, kreisflaeche(2))
```

Ausgabe

```
119.0 12.56636
```

2.2 Typisierung und einfache Datentypen

Der **Datentyp** legt fest, welche Art von Daten vorliegt und welche Operationen erlaubt sind. Mit `type(42)` fragst du ihn ab, das Ergebnis ist `<class 'int'>`. In Python ist jeder Wert ein Objekt, „primitive“ Typen wie in Java gibt es nicht.

Typisierung: statisch = Prüfung beim Kompilieren (Java, C#), **dynamisch** = erst beim Ausführen (Python, JavaScript), **graduell** (im Kurs) = Python mit Typangaben, die `ty` vor dem Ausführen prüft; Python selbst ignoriert sie.

Typ	Beispiele	Wichtig	Java
`int`	`42`, `-7`, `1_000_000`	beliebig groß, kein Überlauf (`2 ** 100` geht)	byte/short/int/long
`float`	`3.14`, `-0.5`	64 Bit, 15–16 Stellen genau; `0.1 + 0.2 == 0.3` ergibt `False`	double
`bool`	`True`, `False`	großgeschrieben	boolean
`str`	`"Hallo"`, `"A"`	unveränderlich; ein Zeichen ist ein str der Länge 1, einen eigenen Typ `char` gibt es nicht	String, char
`None`	`None`	„kein Wert“, nicht 0 und nicht `""`: `telefon: str | None = None`	null

Faustregel: `int` für ganze Zahlen (Anzahl), `float` für Messwerte, `str` für alles, womit man nicht rechnet (PLZ, Telefonnummer; eine führende 0 bliebe sonst nicht erhalten). Für Geld nimmt man `decimal.Decimal`.

```
print(type(42), type(3.0), type("3"))
print(type(True), type(None))
print(2 ** 100)
print(0.1 + 0.2, 0.1 + 0.2 == 0.3)
```

Ausgabe

```
<class 'int'> <class 'float'> <class 'str'>
<class 'bool'> <class 'NoneType'>
1267650600228229401496703205376
0.30000000000000004 False
```

```
# Zeichen ↔ Unicode-Nummer
print(ord("A"))          # A = 65 = 0100 0001
print(chr(66))           # 66 = B
print(chr(ord("A") + 1)) # nächster Buchstabe
print(ord("☺"))          # passt nicht mehr in 8
Bit
```

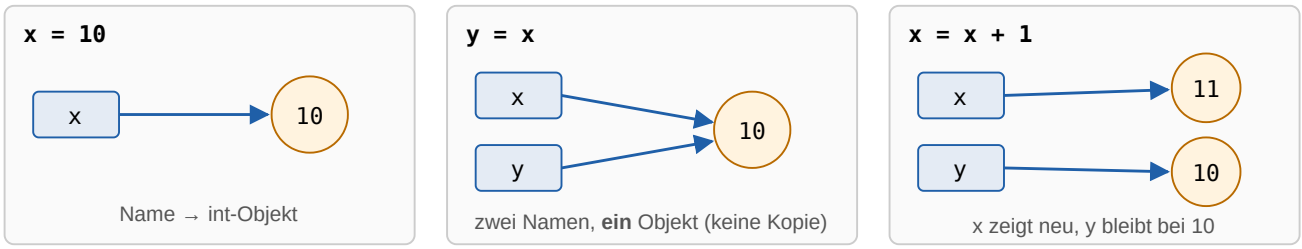
Ausgabe

```
65
B
B
9786
```

2.3 Variablen: Namen zeigen auf Werte

Eine Variable ist ein **Name, der auf einen Wert zeigt**. Stell sie dir wie ein Namensschild vor, das an einem Objekt hängt, nicht wie eine Box. Der Typ gehört zum Wert, nicht zum Namen. Eine Variable entsteht bei der ersten Zuweisung, eine Deklaration wie in Java gibt es nicht. `=` bedeutet „zeigt ab jetzt auf“. Bei `zahl = zahl + 1` rechnet Python zuerst die

rechte Seite aus und hängt dann das Schild um. Den alten Wert gibt Python automatisch frei, wenn kein Name mehr auf ihn zeigt.



Zuweisung hängt ein Namensschild an ein Objekt. Sie kopiert nichts und verändert keinen Wert.

```
alter: int = 25            # Typangabe: Versprechen an ty
breite, hoehe = 1920, 1080 # Mehrfachzuweisung
x = y = 0                  # zwei Namen, ein Wert
a, b = 1, 2
a, b = b, a                # Tauschen ohne Hilfsvariable
wert = 3,5                 # Falle: Komma statt Punkt!
print(alter, breite, hoehe, x, y, a, b)
print(wert, type(wert))
```

Ausgabe

```
25 1920 1080 0 0 2 1
(3, 5) <class 'tuple'>
```

Typangaben (`name: typ = wert`) ignoriert Python beim Ausführen. `ty` prüft sie aber vorher, z. B. meldet es bei `alter: int = "25"` den Fehler `error[invalid-assignment]`. Merksatz der Folie: „Die Typangabe ist ein Versprechen. ty passt auf, dass Sie es halten.“

Tauschen mit `a, b = b, a` funktioniert, weil Python zuerst die ganze rechte Seite auswertet.

NameError: typische Ursachen

Ursache	Beispiel
Der Variable wurde nie ein Wert zugewiesen	`print(summe)` ohne vorheriges `summe = …`
Falsche Groß- und Kleinschreibung	`zahl = 5`, dann `print(Zahl)`
Nur eine Typangabe, aber kein Wert	`anzahl: int`, dann `print(anzahl)`
(Gültigkeit) Der Block mit der Zuweisung wurde übersprungen	`if a > 5: b = 5` mit `a = 3`, dann `print(b)`

Gültigkeitsbereich: Variablen, die in einem `if`- oder Schleifenblock entstehen, gibt es danach weiterhin. Erst Funktionen haben einen eigenen Gültigkeitsbereich (Kap. 4). Wurde der Block aber übersprungen, existiert die Variable nicht.

Namensregeln und Konstanten

Schreibweise	wofür	Beispiel
`snake_case`	Variablen, Funktionen, Methoden, Dateien	`netto_preis`, `berechne_summe()`
`UPPER_SNAKE_CASE`	Konstanten	`MWST_SATZ`
`PascalCase`	Klassen	`Konto`, `Lagerartikel`
camelCase / kebab-case	camelCase ist Java/JS-Stil; kebab-case ist in Python **ungültig** (`-` ist Minus)	`nettoPreis` (unüblich), `netto-preis` (Fehler)

Ein Name besteht aus Buchstaben, Ziffern und `_`. Er darf nicht mit einer Ziffer beginnen und kein Schlüsselwort sein. Umlaute vermeidet man (`groesse`), die Regeln prüft `ruff`. **Konstanten:** Ein `final` wie in Java gibt es nicht. Mit `from typing import Final` und `MWST_SATZ: Final = 0.19` meldet `ty` jede Neuzuweisung. Python selbst verhindert sie zur Laufzeit aber nicht.

2.4 Operatoren

Kategorie	Operatoren	Ergebnis
arithmetisch	`+ - * / // % **`	Zahl
Zuweisung	`= += -= *= /= //= %=`	Name zeigt auf neuen Wert
Vergleich	`== != < > <= >=`	immer `bool`
logisch	`and or not` (Java: `&& || !`)	Wahrheitswert
Text	`+` verketten, `*` wiederholen, `in` enthalten	`str` bzw. `bool`

Die drei Arten zu teilen: `/`, `//`, `%`

Operator	Bedeutung	Beispiele	Ergebnistyp
`/`	echte Division	`7 / 2` → `3.5`, `6 / 3` → `2.0`	**immer float**
`//`	ganzzahlige Division, rundet **ab** (zur kleineren Zahl hin)	`7 // 2` → `3`, `-7 // 2` → `-4` (Java: −3)	int (bei int-Operanden)
`%`	Rest der Division (Modulo)	`7 % 3` → 1, `8 % 3` → 2, `9 % 3` → 0	int
`**`	Potenz, bindet am stärksten	`2 ** 10` → 1024	int/float

Typische Anwendungen für `%`: `zahl % 2 == 0` prüft, ob eine Zahl gerade ist, und `jahr % 4 == 0` prüft die Teilbarkeit durch 4. Die Rangfolge ist **Klammern** → `**` → **Punkt (`* / // %`)** → **Strich (`+ -`)** → **Vergleiche** → `not` → `and` → `or`.

```
print(7 / 2, 6 / 3, 7 // 2, -7 // 2)
print(7 % 3, 8 % 3, 9 % 3, 13 % 2 == 0)
print(2 + 3 * 4, (2 + 3) * 4)
print(5 + 2 / 2, (5 + 2) / 2)   # Semantikfehler!
alter = 30
print(alter >= 18, alter == "30")
print(18 <= alter <= 65, alter % 2 == 0)
```

Ausgabe

```
3.5 2.0 3 -4
1 2 0 False
14 20
6.0 3.5
True False
True True
```

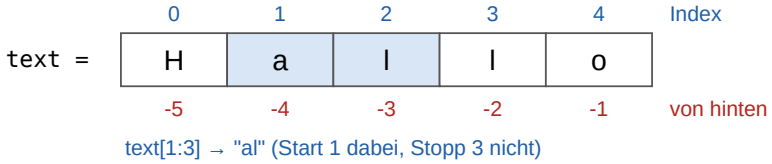
```
# Mini-Aufgabe der Folie
x = 4
x *= 2   # 8
x -= 3   # 5
x //= 2  # 2
print(x)
y = 5
++y      # zweimal Vorzeichen +, ändert nichts
print(y)
print("Python" == "Py" + "thon")
```

Ausgabe

```
2
5
True
```

- **Kein `x++`:** Das ist ein `SyntaxError`. `++x` läuft zwar, ist aber nur zweimal das Vorzeichen + und ändert nichts (`ruff` meldet B002). Zum Hochzählen schreibt man `x += 1`.
- **`==` vergleicht den Inhalt**, auch bei Strings. Ein `.equals()` wie in Java braucht man nicht. `is` prüft, ob es *dasselbe Objekt* ist, und wird nur für `None` verwendet: `if x is None:`. Verkettet: `18 <= alter <= 65` heißt `18 <= alter and alter <= 65`.

2.5 Strings und f-Strings



text[1:3] → "al" (Start 1 dabei, Stopp 3 nicht)

Index ab 0, negative Indizes zählen von hinten, beim Slicing gehört der Stopp-Index nicht dazu.

```
text = "Hallo"
print(len(text), text[0], text[-1], text[1:3])
print("ll" in text, "python".upper())
print("Py" + "thon", "ab" * 3)
```

Ausgabe

```
5 H o al
True PYTHON
Python abababab
```

```
name, alter, preis = "Ada", 20, 12.5
print(f"{name} ist {alter} Jahre alt")
print(f"Preis: {preis:.2f} €")
print(f"Nächstes Jahr: {alter + 1}")
```

Ausgabe

```
Ada ist 20 Jahre alt
Preis: 12.50 €
Nächstes Jahr: 21
```

In einem f-String setzt Python Werte direkt in den Text ein, in den `{}` darf auch gerechnet werden. `:.2f` formatiert auf zwei Nachkommastellen. Strings sind unveränderlich. `text[0] = "h"` gibt deshalb einen TypeError. *(Ergänzung)*

2.6 Typumwandlung, Rundung und Geld

Zum Umwandeln ruft man den Typ wie eine Funktion auf. Automatisch wandelt Python **nur zwischen Zahlen** um, und zwar in der Richtung bool → int → float (`5 + 2.0` → `7.0`, `True + 1` → `2`). Text und Zahl mischt Python nie, das gibt einen `TypeError` („stark typisiert: Python rät nicht“).

Ausdruck	Ergebnis	Achtung
`int(9.99)`	`9`	schneidet ab, rundet nicht
`float(5)` / `str(42)`	`5.0` / `"42"`	
`int("42")` / `int("zwei")`	`42` / `ValueError`	`input()` gibt `str`
`round(2.675, 2)` / `round(9.5)`	`2.67` / `10`	float-Ungenauigkeit
`"Alter: " + 36`	`TypeError`	besser: f-String

```
from decimal import Decimal

print(int(19.99 * 100))   # Cent weg!
print(round(19.99 * 100)) # runden
print(int(Decimal("19.99") * 100))
# Decimal als TEXT übergeben
print(5 + 2.0, True + 1)
print(int("12") + int("3"), "12" + "3")
```

Ausgabe

```
1998
1999
1999
7.0 2
15 123
```

2.7 Syntax vs. Semantik: vier Fehler, drei Prüfer

Die **Syntax** ist die Grammatik, also die Form des Codes. Die **Semantik** ist seine Bedeutung, also was er bewirkt. Code kann syntaktisch korrekt und trotzdem semantisch falsch sein.

Fehlerart	Beispiel	ruff check	ty check	uv run
Syntaxfehler	`print("Hallo"`	× findet ihn	(nicht nötig)	startet gar nicht
Typfehler	`"Alter: " + alter` mit `alter = 36`	✓ ok	× `unsupported-operator`	TypeError
Laufzeitfehler	`summe / anzahl` mit `anzahl = 0`	✓	✓	× ZeroDivisionError
Logikfehler	`5 + 2 / 2` statt `(5 + 2) / 2`	✓	✓	✓ läuft, liefert aber 6.0 statt 3.5

Einen Logikfehler findet nur, wer das Ergebnis nachprüft, zum Beispiel mit einem Test (Kap. 4).

```
summe = 10
anzahl = 0
print("Start")
durchschnitt = summe / anzahl
print(durchschnitt)
```

Ausgabe

```
Start
Traceback … (gekürzt)
ZeroDivisionError: division by zero
```

Einen Traceback liest du von unten nach oben:

1. Die **letzte Zeile** sagt, was passiert ist: Fehlerart und Meldung.
2. Darüber steht, **wo** es passiert ist: Datei und Zeile.
3. Die Markierung `^` zeigt auf die genaue Stelle.

„Start“ wird noch ausgegeben: Bei einem Laufzeitfehler laufen die Zeilen davor, bei einem Syntaxfehler läuft gar nichts.

Typische Fehler und Fallen

- `/` liefert **immer** float (`6 / 3` → `2.0`), und `-7 // 2` ergibt **−4**.
- `5 + 2 / 2` ergibt 6.0 und nicht 3.5, denn Punkt geht vor Strich. Das ist ein Logikfehler **ohne** Fehlermeldung.
- `0.1 + 0.2 == 0.3` ergibt `False`, und `int(19.99 * 100)` ergibt 1998. Floats nie mit `==` auf „krumme“ Werte prüfen, für Geld `Decimal("…")` nehmen.

Das musst du können

☐ Typ und Wert von Ausdrücken mit `/ // % **` vorhersagen, auch mit negativen Zahlen und Klammern
☐ String-Index, Slicing und f-Strings mit `:.2f` anwenden; die vier Fehlerarten erkennen
☐ Ergebnisse von `int()`, `str()`, `round()` vorhersagen; Docstring (Google-Stil) und `Final`-Konstante schreiben

VL3 Kontrollstrukturen und Listen

3.1 Drei Grundformen und Blöcke

Grundform	Was passiert?	Python
Sequenz	Die Anweisungen laufen nacheinander von oben nach unten.	jede Zeile
Selektion	Eine Bedingung wird geprüft und ein Weg gewählt.	`if`, `elif`, `else`, `match`
Iteration	Etwas wird wiederholt, solange eine Bedingung gilt.	`while`, `for`

Ein `:` öffnet einen **Block**. Alle folgenden Zeilen, die 4 Leerzeichen weiter eingerückt sind, gehören dazu. Der Block endet, sobald eine Zeile wieder weniger eingerückt ist. Ein Block hat **keinen eigenen Gültigkeitsbereich**.

Vorab: for-Schleifen ganz langsam (mit Rundentabelle)

In einem Satz: Eine `for`-Schleife sagt „Mach dasselbe noch einmal, für jedes Ding in dieser Reihe, eins nach dem anderen.“ Stell dir eine Kassiererin vor: Sie nimmt eine Ware vom Band (**eine Runde**), scannt sie, legt sie weg und nimmt die nächste. Ist das Band leer, sagt sie die Summe an. Bei `for preis in preise:` ist `preis` die Hand, in der in jeder Runde eine andere Ware liegt. Der einzige Trick für „Was gibt das aus?“: Schreib nach jeder Runde auf, welchen Wert jede Variable hat. Das ist eine **Rundentabelle**. *(Ergänzung, Details zu `for` und `range` in 3.7)*

```
preise = [3, 5, 2]
summe = 0
for preis in preise:
    summe = summe + preis
    print("nach", preis, "ist summe", summe)
print("Gesamt:", summe)
```

Ausgabe

```
nach 3 ist summe 3
nach 5 ist summe 8
nach 2 ist summe 10
Gesamt: 10
```

Runde	`preis`	`summe` vorher	Rechnung	`summe` nachher
Start	–	–	`summe = 0`	0
1	3	0	0 + 3	3
2	5	3	3 + 5	8
3	2	8	8 + 2	10
danach	–	–	`print`	10

Drei Teile, jeder an seiner festen Stelle: (1) **vor** der Schleife der Startwert (`summe = 0`, bei einem Produkt `1`), (2) **in** der Schleife dazuzählen, (3) **nach** der Schleife das Ergebnis benutzen (`print` oder `return`). Der Wert vom Ende einer Runde ist der Wert am Anfang der nächsten. Zählen ist dasselbe Muster mit `anzahl += 1`, aber nur im `if`-Zweig.

`range(start, stop, schritt)`: Start ist dabei, Stopp nicht. `range(2, 9, 3)` liefert 2, 5, 8 (11 wäre zu groß). Mit `print(i, i * i)` in der Schleife und `print("danach:", i)` dahinter erscheinen `2 4`, `5 25`, `8 64` und `danach: 8`: Nach der Schleife behält `i` den letzten Wert.

Falle: Steht `summe = 0` **in** der Schleife, wird die Kasse in jeder Runde wieder auf null gestellt:

```
preise = [3, 5, 2]
for preis in preise:
    summe = 0  # FEHLER: in der Schleife
    summe = summe + preis
print("Gesamt:", summe)
```

Ausgabe

```
Gesamt: 2
```

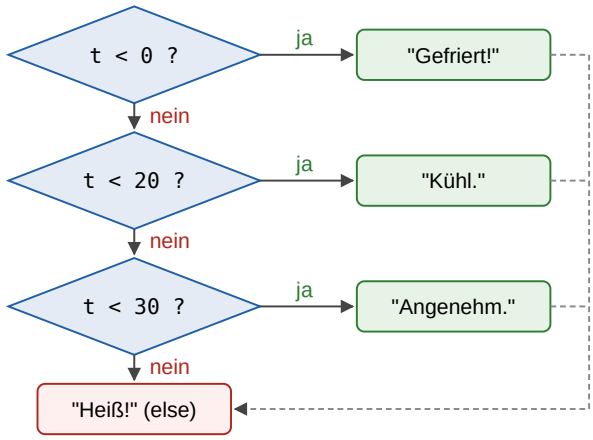
Runde 1: 0 → 3, Runde 2: wieder 0 → 5, Runde 3: wieder 0 → 2. Genauso falsch ist `return` in der Schleife (nach Runde 1 ist Schluss). Beides siehst du an der Einrückung.

3.2 Bedingte Anweisungen: `if`, `elif`, `else`

Um die Bedingung stehen keine Klammern, am Ende steht ein Doppelpunkt. `else` steht auf derselben Einrückungsebene wie `if`. Bei `if`–`else` läuft **genau einer** der beiden Blöcke. Mit `elif` sparst du dir verschachtelte `if`s. Python prüft die Bedingungen **der Reihe nach** und hört beim **ersten Treffer** auf, die übrigen Zweige werden dann gar nicht mehr geprüft.

```
temperatur = 25
if temperatur < 0:
    print("Gefriert!")
elif temperatur < 20:
    print("Kühl.")
elif temperatur < 30:
    print("Angenehm.")
else:
    print("Heiß!")
```

Ausgabe
```
Angenehm.
```



danach geht es nach dem Block weiter

Reihenfolge zählt: Bei Bereichsprüfungen sortierst du die Bedingungen von oben nach unten. Steht in der Notenaufgabe `punkte >= 50` an erster Stelle, bekommt auch jemand mit 95 Punkten „ausreichend“, und zwar ohne Fehlermeldung (Logikfehler). Grenzen testest du mit einem Wert genau auf der Grenze und einem knapp darunter (90 und 89).

3.3 `match` und der bedingte Ausdruck

```
note = "E"
match note:
    case "A" | "B" | "C":
        print("Bestanden")
    case "D" | "E" | "F":
        print("Nicht bestanden")
    case _:
        print("Ungültige Eingabe")
```

Ausgabe
```
Nicht bestanden
```

`match` gibt es seit Python 3.10, es entspricht Javas `switch`. Es vergleicht einen Wert mit **festen** Fällen, und der erste passende Fall gewinnt. Ein `break` braucht man nicht, es gibt **kein Durchfallen** in den nächsten Fall. `case _:` passt immer und ist der Standardfall (Java: `default`). Mehrere Werte verbindest du mit `|`.

`if`–`elif` bleibt der Normalfall, weil nur damit Bereiche wie `note < 3` geprüft werden können.

Bedingter Ausdruck (Kurzform von if–else, Java: `b ? x : y`): `status = "offen" if betrag > 0 else "bezahlt"`. Er wählt einen von zwei Werten und sollte nicht verschachtelt werden. `ruff` schlägt ihn selbst vor, wenn ein if–else nur eine Variable setzt.

3.4 Wahrheitswerte beliebiger Werte

In einer Bedingung darf jeder Wert stehen. Python fragt dann: „Ist da etwas?“

gilt als falsch	gilt als wahr
`False`, `None`	`True`
`0`, `0.0`	jede andere Zahl, **auch `-1`**
`""` (leerer String)	`"0"`, `" "`: jeder nicht leere String
`[]` (leere Liste)	`[0]`: jede nicht leere Liste

```
warenkorb: list[str] = []
if warenkorb:
    print(f"{len(warenkorb)} Artikel")
else:
    print("Der Warenkorb ist leer")
print(bool("0"), bool(-1), bool([0]), bool(""))
```

Ausgabe
```
Der Warenkorb ist leer
True True True False
```

3.5 Logische Operatoren, Rangfolge, Kurzschluss

`a`	`b`	`a and b`	`a or b`	`not a`	XOR: `a != b`
True	True	True	True	False	False
True	False	False	True	False	True
False	True	False	True	True	True
False	False	False	False	True	False

Rangfolge: Klammern → Vergleiche → `not` → `and` → `or`. `a or b and not c` bedeutet also `a or (b and (not c))`. Wenn `and` und `or` in einer Bedingung vorkommen, setzt du Klammern. Die Gesetze der Logik gelten auch hier: **kommutativ** (die Reihenfolge der Operanden ist egal), **assoziativ** (die Klammerung bei gleichen Operatoren ist egal) und **distributiv** (`a and (b or c)` = `(a and b) or (a and c)`).

```
# Rabatt: Studierende ODER ab 65, beide NUR mit
Ausweis
ist_student, alter, hat_ausweis = True, 22, False
print(ist_student or alter >= 65 and hat_ausweis)
print((ist_student or alter >= 65) and hat_ausweis)
```

Ausgabe

```
True
False
```

```
# Kurzschluss: Absicherung nach vorn!
anzahl, umsatz = 0, 0.0
print(anzahl > 0 and umsatz / anzahl > 100)
print(umsatz / anzahl > 100 and anzahl > 0)
```

Ausgabe

```
False
Traceback … (gekürzt)
ZeroDivisionError: float division by zero
```

Kurzschlussauswertung: Python wertet von links nach rechts aus und hört auf, sobald das Ergebnis feststeht. Ist bei `and` die linke Seite `False`, wird die rechte gar nicht mehr ausgewertet. Die Absicherung (`anzahl > 0`) gehört deshalb nach vorn.

Bedingung umkehren: Entweder du schreibst `not (…)` um die ganze Bedingung, oder du kehrst jede Teilbedingung um und tauschst `and` und `or`: Das Gegenteil von `bestellwert >= 50 or ist_premium` ist `bestellwert < 50 and not ist_premium`, das Gegenteil von `hat_ticket and not ist_gesperrt` ist `not hat_ticket or ist_gesperrt`.

3.6 Übungen Teil 1: Stromkosten und Parkgebühren

```
verbrauch_kwh = 180   # sonst: int(input(...))
# Staffel: jede Stufe gilt nur für IHRE kWh
if verbrauch_kwh <= 100:
    kosten = verbrauch_kwh * 0.25
elif verbrauch_kwh <= 300:
    kosten = 100 * 0.25 + (verbrauch_kwh - 100) *
0.22
else:
    kosten = 100 * 0.25 + 200 * 0.22 +
(verbrauch_kwh - 300) * 0.19
kosten += 12.90                 # Grundpreis
if verbrauch_kwh <= 250:        # Öko-Bonus
    kosten -= 5
kosten = min(kosten, 100)       # Deckel 100 €
print(f"Stromkosten: {kosten:.2f} Euro")
```

Ausgabe

```
Stromkosten: 50.50 Euro
```

```
GRUNDGEBUEHR, HOECHSTPREIS = 1.50, 50.00  # Lösung:
Final
fahrzeug, stunden = "Transporter", 6
kosten, gueltig = 0.0, True
match fahrzeug:
    case "PKW":
        kosten = stunden * 2.00
        if stunden >= 10:
            kosten *= 0.90
    case "Motorrad":
        kosten = stunden * 1.00
    case "Transporter":
        kosten = stunden * 3.50
        if stunden >= 5:
            kosten *= 0.80
    case _:
        print("Fehler: Ungültiger Fahrzeugtyp!")
        gueltig = False
if gueltig:  # Rabatt → Grundgebühr → Deckel
    kosten = min(kosten + GRUNDGEBUEHR,
HOECHSTPREIS)
    print(f"Gesamtkosten: {kosten:.2f} Euro")
```

Ausgabe

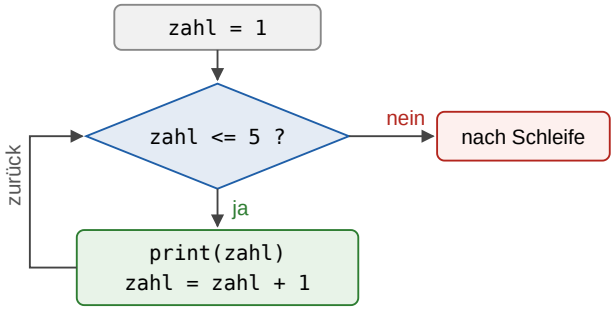
```
Gesamtkosten: 18.30 Euro
```

Nach der offiziellen Musterlösung: Bei 180 kWh ergibt sich 25,00 + 17,60 + 12,90 − 5,00 = 50,50 €. Bei 6 Stunden Transporter ergibt sich 21,00 · 0,8 + 1,50 = 18,30 €. Weitere Übung aus der Musterlösung, das Schaltjahr: `jahr % 4 == 0 and (jahr % 100 != 0 or jahr % 400 == 0)`.

3.7 Schleifen: `while` und `for`

`while` prüft zuerst und läuft dann. Ist die Bedingung schon am Anfang falsch, läuft die Schleife **kein einziges Mal**. Im Block muss sich etwas ändern, das die Bedingung irgendwann falsch macht, sonst entsteht eine **Endlosschleife**. Abbrechen kannst du sie im Terminal mit Strg+C. Die Bedingung ist eine *Laufbedingung*: Die Schleife läuft, *solange* sie wahr ist.

`while` nimmt man, wenn die Zahl der Durchläufe vorher **unbekannt** ist (z. B. „Bis wann hat sich das Guthaben verdoppelt?“).



Fehlt „zahl = zahl + 1“: Endlosschleife

```
zahl = 1
while zahl <= 5:
    print(f"Zahl: {zahl}")
    zahl = zahl + 1
print("zahl ist jetzt", zahl)
```

Ausgabe

```
Zahl: 1
Zahl: 2
Zahl: 3
Zahl: 4
Zahl: 5
zahl ist jetzt 6
```

```
guthaben, jahre = 1000.0, 0
while guthaben < 2000:      # Anzahl unbekannt
    guthaben *= 1.05
    jahre += 1
print(f"Nach {jahre} Jahren: {guthaben:.2f} Euro")
```

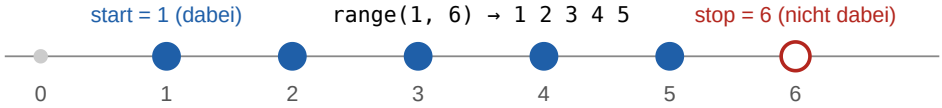
Ausgabe

```
Nach 15 Jahren: 2078.93 Euro
```

Ein `do – while` gibt es nicht. Für „mindestens einmal“ schreibst du `while True:` mit einem `break` im Block. Das ist das typische Muster für eine Eingabeprüfung:

```
while True:
    eingabe = input("Menge (1-99): ")
    if eingabe.isdigit() and 1 <= int(eingabe) <= 99:
        break
    print("Ungültig, bitte noch einmal.")
```

`for x in folge:` läuft für **jedes Element** einmal durch. Einen Zähler oder `i++` brauchst du dafür nicht. Zahlenfolgen erzeugst du mit `range(start, stop, schritt)`: **start ist dabei, stop nicht.**



Weitere Folgen: `range(5)` → 0 1 2 3 4 (beginnt bei 0) · `range(2, 11, 2)` → 2 4 6 8 10 · `range(5, 0, -1)` → 5 4 3 2 1 (rückwärts, ohne die 0) · für „1 bis n“: `range(1, n + 1)`.

```
n = 7
ergebnis = 1
for i in range(1, n + 1):  # n + 1: stop gehört
nicht dazu
    ergebnis = ergebnis * i
print(f"Fakultät von {n} ist: {ergebnis}")
```

Ausgabe

```
Fakultät von 7 ist: 5040
```

```
a, b = 0, 1
for _ in range(10):        # _ = Variable wird
nicht gebraucht
    print(a, end=" ")
    a, b = b, a + b        # gleichzeitig
weiterschieben!
print()
```

Ausgabe

```
0 1 1 2 3 5 8 13 21 34
```

Java-Zählschleife und Python im Vergleich:

Java

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

```
for i in range(1, 6):
    print(i)
```

3.8 `break`, `continue` und typische Schleifenfehler

```
for i in range(1, 11):
    if i == 5:
        break          # Schleife sofort verlassen
    print(f"i = {i}")
print("Nach der Schleife")
```

Ausgabe

```
i = 1
i = 2
i = 3
i = 4
Nach der Schleife
```

```
for i in range(1, 6):
    if i == 3:
        continue       # nur diesen Durchlauf
auslassen
    print(f"i = {i}")
```

Ausgabe

```
i = 1
i = 2
i = 4
i = 5
```

Fehler	Beispiel	Folge
Endlosschleife	Zähler wird nicht verändert oder `zahl += 1` ist nicht eingerückt	hört nie auf (Strg+C)
Off-by-one	`range(1, 5)` für 1 bis 5	eine Runde zu wenig (endet bei 4)
falscher Startwert	`zahl = 10`, dann `while zahl <= 5`	läuft nie, **ohne** Meldung
falsche Schrittweite	`zahl += 2` statt `+= 1`	überspringt Werte, ohne Meldung

Ein `else` nach einer Schleife läuft nur, wenn die Schleife **ohne** `break` endet. Das braucht man selten.

3.9 Listen

Eine Liste ist eine geordnete Sammlung unter einem Namen, auf die einzelnen Werte greifst du über den **Index** zu. Anders als ein Java-Array hat sie **keine feste Länge**. Guter Stil ist ein Typ pro Liste. Automatische Standardwerte gibt es nicht. Du legst also entweder eine leere Liste an und hängst mit `append` an oder schreibst `[0] * n`.

```
noten = [2, 1, 3, 2, 4]
leer: list[int] = []      # Typangabe bei leerer
Liste
nullen = [0] * 3
leer.append(7)
noten[0] = 1              # ändern
print(noten, leer, nullen)
print(noten[0], noten[-1], noten[1:3])
print(len(noten), sum(noten), min(noten),
max(noten))
print(3 in noten, sorted(noten), noten)
print(noten[5])
```

Ausgabe

```
[1, 1, 3, 2, 4] [7] [0, 0, 0]
1 4 [1, 3]
5 11 1 4
True [1, 1, 2, 3, 4] [1, 1, 3, 2, 4]
Traceback … (gekürzt)
IndexError: list index out of range
```

```
tage = ["Mo", "Di", "Mi"]
werte = [12.5, -3.0, 7.0]
for i in range(len(tage)):          # über Index
    print(i, tage[i])
for wert in werte:                  # Python-Weg
    print(wert, end=" ")
print()
for i, tag in enumerate(tage):      # Index + Wert
    print(i, tag, end=" | ")
print()
for tag, wert in zip(tage, werte, strict=True):
    print(f"{tag}: {wert}")
```

Ausgabe

```
0 Mo
1 Di
2 Mi
12.5 -3.0 7.0
0 Mo | 1 Di | 2 Mi |
Mo: 12.5
Di: -3.0
Mi: 7.0
```

`sorted(liste)` liefert eine sortierte **Kopie**, die Originalliste bleibt unverändert. `zip(..., strict=True)` geht zwei Listen im Gleichschritt durch und meldet einen Fehler, wenn sie unterschiedlich lang sind. `ruff` verlangt dieses `strict=`.

Mehrdimensionale Listen und die Verweis-Falle

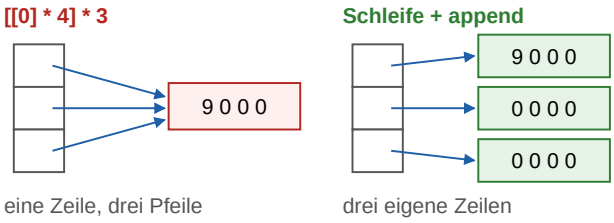
Eine Tabelle ist eine Liste von Listen. `matrix[1][2]` heißt Zeile 1, Spalte 2. Durchlaufen wird sie mit zwei verschachtelten Schleifen.

```
falsch = [[0] * 4] * 3     # 3 Verweise auf EINE Zeile
falsch[0][0] = 9
print(falsch)

richtig = []
for _ in range(3):         # jede Zeile neu bauen
    richtig.append([0] * 4)
richtig[0][0] = 9
print(richtig)
```

Ausgabe

```
[[9, 0, 0, 0], [9, 0, 0, 0], [9, 0, 0, 0]]
[[9, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```



Das * 3 kopiert nur die Verweise, nicht die Zeile selbst (wie zwei Namensschilder an einem Objekt, Kap. 2.3).

Tupel `(3, 4)` sind wie Listen, aber **unveränderlich**: `punkt[0] = 5` gibt einen `TypeError`. Man nimmt sie für feste Zusammenstellungen wie Koordinaten oder mehrere Rückgabewerte.

3.10 Übungen Teil 2 (Musterlösungen)

```
temperaturen = [-3.5, -1.0, -7.2, -0.5]
maximum = temperaturen[0]  # nicht mit 0 starten!
summe = 0.0
for t in temperaturen:
    summe += t
    if t > maximum:
        maximum = t
print(maximum, summe / len(temperaturen))
print(max(temperaturen), sum(temperaturen) /
len(temperaturen))
```

Ausgabe

```
-0.5 -3.05
-0.5 -3.05
```

```
# Sieb des Eratosthenes bis N
N = 30
ist_prim = [True] * (N + 1)
ist_prim[0] = ist_prim[1] = False
p = 2
while p * p <= N:              # reicht bis √N
    if ist_prim[p]:
        for vielfaches in range(p * p, N + 1, p):
            ist_prim[vielfaches] = False
    p += 1
for zahl, prim in enumerate(ist_prim):
    if prim:
        print(zahl, end=" ")
print()
```

Ausgabe

```
2 3 5 7 11 13 17 19 23 29
```

Weitere Übungen: gerade Zahlen 1–100 zählen (50), Multiplikation durch wiederholte Addition, erste durch 7 teilbare Zahl ab 100 mit `break` (105). Sieb: Folie streicht ab 2·p, Lösung ab p·p; beides richtig.

Typische Fehler und Fallen

- Doppelpunkt vergessen: `SyntaxError: expected ':'`. Einrückung vergessen: `IndentationError`. `else` ohne direkt vorangehendes `if`: `SyntaxError`. Leerzeichen und Tabs nicht mischen.
- Eine nicht eingerückte Zeile gehört nicht mehr zum Block und läuft **immer**.
- Bei `elif` zählt nur der erste Treffer, und `match` kann keine Bereiche prüfen.
- `"0"`, `-1` und `[0]` sind **wahr**. `0`, `""`, `[]` und `None` sind falsch.
- Ohne Klammern bindet `and` stärker als `or`. Die Absicherung (`x != 0 and …`) gehört nach vorn.
- `range(1, 5)` endet bei 4. Fakultät also mit `range(1, n + 1)`.
- Fibonacci falsch: `a = b` und danach `b = a + b` rechnet schon mit dem neuen `a`. Richtig ist `a, b = b, a + b`.
- Maximum mit 0 starten ist falsch, wenn alle Werte negativ sind. Starte mit `liste[0]`.

Das musst du können

☐ die Ausgabe von if/elif/else, `match` und bedingten Ausdrücken für gegebene Werte vorhersagen
☐ Staffelpreis- und Gebührenaufgaben mit `if`/`elif`/`match` und `Final`-Konstanten programmieren
☐ zusammengesetzte Bedingungen mit Rangfolge, Kurzschluss und Umkehrung auswerten
☐ `while`- und `for`-Schleifen in einer Trace-Tabelle verfolgen und die Folgen von `range` vorhersagen
☐ `break` und `continue` unterscheiden und Endlosschleife, Off-by-one, falschen Startwert und falsche Schrittweite erkennen
☐ Listen anlegen, indizieren, slicen und mit Index, direkt, `enumerate` und `zip` durchlaufen
☐ Summe, Zähler, Maximum, Fibonacci und das Sieb mit Schleifen programmieren
☐ die Verweis-Falle erklären und Liste und Tupel unterscheiden

VL4 Funktionen und Rekursion

4.1 Funktionen definieren und aufrufen

Eine **Funktion** ist ein abgeschlossener Codeblock mit einem Namen, der eine bestimmte Aufgabe erfüllt. Sie bekommt Eingaben (Parameter), rechnet damit und liefert ein Ergebnis, ähnlich einer kleinen Maschine. Einige kennst du schon: `print()`, `len()`, `int()`, `input()`. Funktionen machen Programme übersichtlich, wiederverwendbar und testbar. `def` legt eine Funktion nur an, ihr Rumpf läuft erst beim **Aufruf**. Weil Python von oben nach unten liest, muss die Definition vor dem ersten Aufruf stehen.

```
def begruessung(name: str) -> None:  # Definition
    print(f"Hallo {name}!")

def addiere(a: int, b: int) -> int:
    summe = a + b
    return summe          # Wert an den Aufrufer

def main() -> None:
    begruessung("Bob")    # Aufruf: Argument "Bob"
    ergebnis = addiere(5, 7)
    print(ergebnis)
    begruessung(42)       # läuft! Nur ty meldet es

if __name__ == "__main__":
    main()
```

Ausgabe

```
Hallo Bob!
12
Hallo 42!
```

Begriff	Bedeutung
Parameter	Platzhalter in der Definition (`name`). In der Funktion verhält er sich wie eine lokale Variable.
Argument	der konkrete Wert beim Aufruf (`"Bob"`)
Signatur	Name, Parameter mit Typen und Rückgabetyp: `def addiere(a: int, b: int) -> int`
Typannotationen	schreiben wir im Kurs **immer**. Python ignoriert sie beim Ausführen, `ty check` meldet z. B. `invalid-argument-type`.

Einstiegspunkt: Ab VL4 steht der Programmablauf in `main()`. `__name__` hat den Wert `"__main__"`, wenn die Datei direkt gestartet wird. Wird sie von einer anderen Datei **importiert**, läuft `main()` nicht los, und ihre Funktionen können wiederverwendet und getestet werden. Das entspricht `public static void main` in Java, ist in Python aber nur eine Konvention.

Kommandozeilenargumente: Nach `import sys` ist `sys.argv` eine Liste von `str`. `sys.argv[0]` ist der Skriptname, die eigentlichen Argumente stehen in `sys.argv[1:]` (Java: `String[] args`).

4.2 Rückgabewerte und `None`

```
def ist_gerade(zahl: int) -> bool:
    return zahl % 2 == 0

def min_max(zahlen: list[int]) -> tuple[int, int]:
    return min(zahlen), max(zahlen)   # ein Tupel

def vorzeichen(x: int) -> str:
    if x > 0:
        return "positiv"
    # x <= 0: kein return → None!

kleinste, groesste = min_max([4, 9, 2, 7])
print(ist_gerade(7), kleinste, groesste)
print(vorzeichen(5), vorzeichen(-3))
print(print("Hi"))    # print liefert None
```

Ausgabe

```
False 2 9
positiv None
Hi
None
```

- Der Rückgabetyp steht nach `->`, jeder Typ ist möglich.
- `return` beendet die Funktion **sofort**, Code danach läuft nicht mehr.
- „Mehrere Rückgabewerte“ sind in Wahrheit **ein Tupel**, das der Aufrufer wieder auspackt.
- **Ohne `return` liefert jede Funktion `None`.** Bei `-> None` darf ein leeres `return` die Funktion vorzeitig beenden.
- Fehlt in einem Zweig das `return`, kommt dort `None` heraus. `ty` meldet `invalid-return-type: Function can implicitly return None`.
- **`print` ist nicht `return`:** Eine Funktion, die ihr Ergebnis nur ausgibt, liefert `None`.

Aufrufstapel (Call Stack)

Bei jedem Aufruf, z. B. `addiere(5, 7)`, legt Python einen neuen **Rahmen** (Stack Frame) an und bindet darin `a = 5` und `b = 7`. Währenddessen wartet `main`. Mit `return` geht 12 zurück an `main`, der Rahmen verschwindet samt `a`, `b` und `summe`, und `main` macht in derselben Zeile weiter.

`break` vs. `return`

Anweisung	verlässt …	danach
`break`	nur die aktuelle (innerste) Schleife	Der Rest der Funktion läuft weiter.
`continue`	nur den aktuellen Schleifendurchlauf	Es geht mit dem nächsten Wert weiter.
`return`	die **ganze Funktion**, auch mitten in einer Schleife	Der Wert geht an den Aufrufer, der Rahmen verschwindet.

```
def mit_break() -> None:
    for i in range(1, 11):
        if i == 5:
            break
        print(i, end=" ")
    print("Schleife beendet.")

mit_break()
```

Ausgabe
```
1 2 3 4 Schleife beendet.
```

```
def mit_return() -> None:
    for i in range(1, 11):
        if i == 5:
            return
        print(i, end=" ")
    print("Schleife beendet.")  # kommt nie dran

mit_return()
print()
```

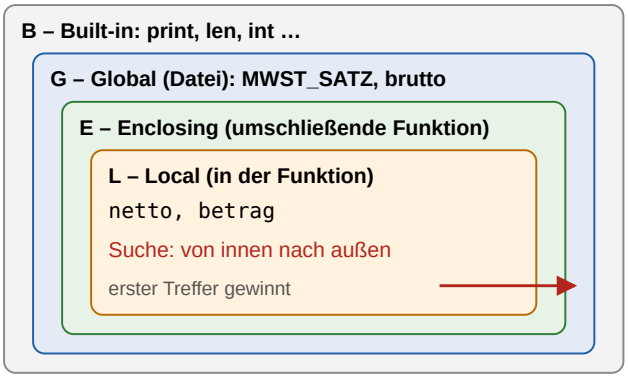
Ausgabe
```
1 2 3 4
```

Folienhinweis (VL4, „break und return im Ablauf 1/2“)

Im Ausgabefeld der Folie stehen nur `2`, `3`, `4` und „Schleife beendet.“, die `1` fehlt (vermutlich abgeschnitten). Richtig ist **1 2 3 4** und dann „Schleife beendet.“, siehe die Ausgabe oben.

4.3 Gültigkeitsbereiche (LEGB)

Variablen, die in einer Funktion entstehen, sind **lokal**. Sie existieren nur, solange die Funktion läuft, und außerhalb gibt es einen `NameError`. Zur Erinnerung: `if` und `for` haben keinen eigenen Gültigkeitsbereich, **Funktionen schon**. Einen Namen sucht Python in der Reihenfolge **LEGB**.




```
MWST_SATZ = 0.19                  # global

def brutto(netto: float) -> float:
    betrag = netto * (1 + MWST_SATZ)  # lesen geht
    return betrag

zaehler = 0

def erhoehe() -> None:
    zaehler += 1   # Zuweisung → lokal!

print(brutto(100.0))
erhoehe()
```

Ausgabe
```
119.0
Traceback … (gekürzt)
UnboundLocalError: cannot access local variable
'zaehler' where it is not associated with a value
```

Eine globale Variable **lesen** geht. Eine **Zuweisung** in der Funktion macht den Namen aber für die *ganze* Funktion lokal. Bei `zaehler += 1` wird dann gelesen, bevor ein Wert da ist, und es gibt einen `UnboundLocalError`. Besser ist es, den Wert als Parameter zu übergeben und das Ergebnis mit `return` zurückzugeben.

4.4 Parameterübergabe: Namen, keine Kopien

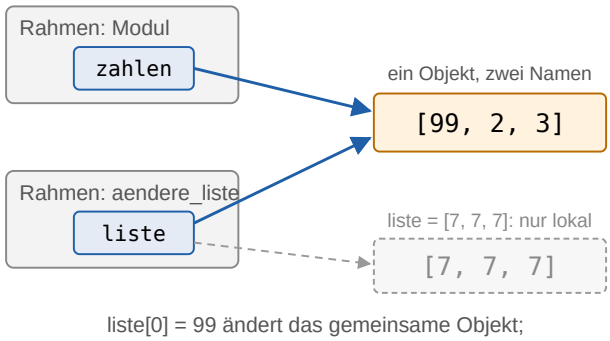
Python übergibt **Verweise auf Objekte**, man nennt das „call by object reference“. Der Parameter zeigt also auf **dasselbe Objekt** wie das Argument. Was das bewirkt, hängt davon ab, ob die Funktion das Objekt **verändert** oder den Namen nur **neu bindet**.

```
def aendere_zahl(wert: int) -> None:
    wert = wert + 1          # nur lokale Neubindung

def aendere_liste(liste: list[int]) -> None:
    liste[0] = 99            # verändert DAS Objekt
    liste = [7, 7, 7]        # nur lokale Neubindung

zahl = 5
zahlen = [1, 2, 3]
aendere_zahl(zahl)
aendere_liste(zahlen)
print(zahl, zahlen)
```

Ausgabe
```
5 [99, 2, 3]
```



liste[0] = 99 ändert das gemeinsame Objekt;

Was passiert in der Funktion?	Beispiel	Außen sichtbar?
Name neu binden	`wert = wert + 1`, `liste = [7, 7, 7]`	nein, das bleibt lokal
Objekt verändern	`liste[0] = 99`, `liste.append(4)`	**ja**, das ist ein **Nebeneffekt** (Java verhält sich bei Arrays genauso)

Falle: veränderbarer Standardwert. Ein Standardwert wird **einmal beim Definieren** erzeugt und dann bei jedem Aufruf wiederverwendet. Bei einer Liste als Standardwert teilen sich also alle Aufrufe dieselbe Liste (`ruff` meldet B006). Richtig ist `None` als Standardwert, die Liste legst du erst im Rumpf an.

```
def falsch(wert: int, liste: list[int] = []) ->
list[int]:
    liste.append(wert)
    return liste

print(falsch(1))
print(falsch(2))     # erwartet [2]
```

Ausgabe
```
[1]
[1, 2]
```

```
def anhaengen(wert: int, liste: list[int] | None =
None) -> list[int]:
    if liste is None:
        liste = []
    liste.append(wert)
    return liste

print(anhaengen(1))
print(anhaengen(2))
```

Ausgabe
```
[1]
[2]
```

Gute Funktionen: Funktionen dürfen andere Funktionen aufrufen, so baust du ein Programm aus kleinen Bausteinen zusammen (`quadratsumme` nutzt z. B. `quadriere`). Faustregel: **eine Funktion, eine Aufgabe, und sie passt auf einen Bildschirm.** Dokumentiert wird mit einem Docstring im Google-Stil (Kap. 2.1), den `help(funktion)` anzeigt (Java: Javadoc).

4.5 Standardwerte, Schlüsselwortargumente, `*args` statt Überladung

Java löst „gleiche Aufgabe, andere Eingaben“ mit **Methodenüberladung**. **Python kennt keine Überladung.** Ein Name zeigt immer auf genau ein Objekt, ein zweites `def` mit demselben Namen **überschreibt** also das erste (`ruff`: F811). Ruft man danach die alte Variante auf, gibt es einen `TypeError`.

```
def brutto(netto: float, satz: float = 0.19) ->
float:
    return round(netto * (1 + satz), 2)

print(brutto(100))                    # Standardwert
print(brutto(100, 0.07))              # Position
print(brutto(satz=0.07, netto=100))   # Name
print(brutto())
```

Ausgabe
```
119.0
107.0
107.0
Traceback … (gekürzt)
TypeError: brutto() missing 1 required positional
argument: 'netto'
```

```
def summe(*zahlen: int) -> int:   # sammelt in ein
Tupel
    print(zahlen, end=" → ")
    ergebnis = 0
    for zahl in zahlen:
        ergebnis += zahl
    return ergebnis

print(summe(1, 2, 3, 4))
print(summe())
```

Ausgabe
```
(1, 2, 3, 4) → 10
() → 0
```

So kommen Argumente zu den Parametern: Zuerst ordnet Python die Argumente ohne Namen der Reihe nach zu. Parameter, für die kein Argument kommt, bekommen ihren Standardwert. Benannte Argumente

(Schlüsselwortargumente) finden ihren Parameter über den Namen, die Reihenfolge ist dann egal. Fehlt ein Pflichtparameter, bricht Python schon beim Aufruf ab, die Funktion läuft gar nicht erst los. Parameter **mit** Standardwert müssen **hinter** denen ohne stehen, sonst gibt es `SyntaxError: parameter without a default follows parameter with a default`.

Java	Python
andere Parameteranzahl → Überladung	Standardwerte, `*args`
andere Parametertypen → Überladung	eine Funktion mit `int | float` oder getrennte Namen
Variante ohne Parameter ruft die Variante mit Standardwert auf	Standardwert direkt in der Signatur
Der Compiler wählt die Methode (Compile-Time Binding)	Pro Name gibt es genau eine Funktion.
nur anderer Rückgabetyp → Compilerfehler	Das zweite `def` überschreibt das erste (F811).

Unterschiedliche Aufgaben bekommen besser unterschiedliche Namen: `flaeche_kreis()`, `flaeche_rechteck()`.

4.6 Imperative vs. applikative Programmierung

Ein **Paradigma** ist ein Denkmuster, das bestimmt, wie man Algorithmen formuliert und entwirft. Im Fokus stehen **imperative** und **applikative** Algorithmen. Weitere Paradigmen sind objektorientiert, logisch, genetisch, parallel, neuronal und agentenorientiert.

	Imperativ	Applikativ (funktional)
Denkschema	**Wie** mache ich es?	**Was** ist das Ergebnis?
Grundlage	Anweisungen und Variablen, sequenzielle Ausführung	Anwendung von Funktionen: `f(v1, …, vn) = t(v1, …, vn)` mit einem Term t
Zustand / Variablen	ändert sich ständig	bleibt unverändert, Werte sind konstant
Wiederholung	`for`, `while`	Rekursion
Nebenwirkungen	häufig	unerwünscht, das Ergebnis hängt nur von den Eingaben ab
Beispiel	Summieren mit `for`	Summieren mit Selbstaufruf; Funktionen höherer Ordnung (`map`, `filter`, `reduce`)

```
FAC:                          (imperatives Schema)
  var X, Y: int;
  input X;
  Y := 1;
  while X > 1 do
    Y := Y · X;
    X := X - 1
  od
  output Y.
```

```
# h(p,q,x,y) = if p ∧ q then 4x + 2y else 5x − 3y
fi
def h(p: bool, q: bool, x: int, y: int) -> int:
    return 4 * x + 2 * y if p and q else 5 * x - 3
* y

print(h(True, False, 1, 3))   # 5 - 9
print(h(True, True, 2, 8))    # 8 + 16
```

Ausgabe
```
-4
24
```

Folienfehler (VL4, „Applikative Algorithmen“)

Im Code der Folie steht `else 5 * x - 3`, das **`* y` fehlt**. Richtig ist `5 * x - 3 * y`, so wie in der mathematischen Definition. Mit dem Folien-Code käme `h(True, False, 1, 3) == 2` heraus und nicht die auf der Folie angegebene −4. Die Ausgaben −4 und 24 stimmen nur mit dem korrigierten Code (siehe oben).

4.7 Rekursion

Bei einer **Rekursion** ruft eine Funktion **sich selbst** auf, um ein kleineres Teilproblem derselben Art zu lösen. Das geht so lange weiter, bis ein einfacher Fall erreicht ist. Jede rekursive Funktion hat zwei Pflichtteile:

1. eine **Abbruchbedingung** (Basisfall): Sie legt fest, wann die Rekursion aufhört, und verhindert endlose Rekursion.
2. einen **rekursiven Aufruf** mit einem **vereinfachten** Argument, das sich der Abbruchbedingung nähert.

```python
def fakultaet(n: int) -> int:
    """Berechnet n! rekursiv."""
    if n == 0:                          #
Abbruchbedingung
        return 1
    else:
        return n * fakultaet(n - 1)    # rekursiver
Aufruf

def summe_bis(n: int) -> int:
    return 0 if n == 0 else n + summe_bis(n - 1)

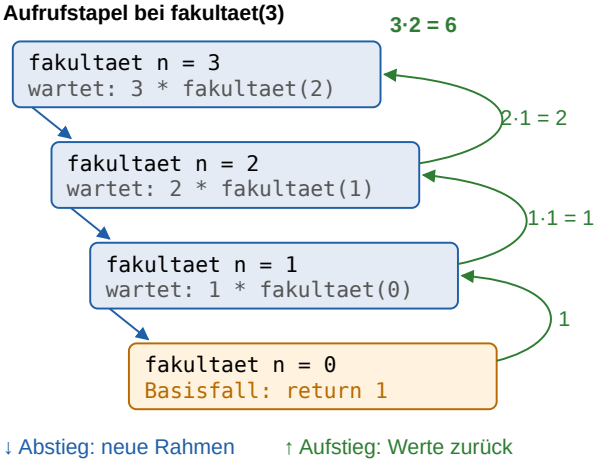
def countdown(n: int) -> None:         # nur Aktion
    if n == 0:
        print("Start!")
    else:
        print(n, end=" ")
        countdown(n - 1)

print(fakultaet(3), fakultaet(5), summe_bis(7))
countdown(3)
```

Ausgabe

```
6 120 28
3 2 1 Start!
```

Aufrufstapel bei fakultaet(3)



Im Speicher bekommt jeder Aufruf einen eigenen Rahmen mit einem eigenen `n`. Beim **Abstieg** wartet jeder Rahmen auf das Ergebnis des nächsten. Erst der Basisfall liefert einen Wert, danach rechnen beim **Aufstieg** alle wartenden Rahmen ihr Ergebnis aus.

Ohne Abbruchbedingung, oder wenn sie nie erreicht wird, wächst der Stapel immer weiter. `fakultaet(-1)` mit `if n == 0` läuft z. B. über −1, −2, … bis zum `RecursionError: maximum recursion depth exceeded` (Java: `StackOverflowError`). `sys.getrecursionlimit()` liefert etwa **1000** Rahmen. Python optimiert Rekursion nicht weg (keine „Tail-Call-Optimierung“), deshalb bricht auch `fakultaet(5000)` ab. Für große `n` löst man die Aufgabe **iterativ**, in echten Programmen mit `math.factorial()`.

```python
def fakultaet_iterativ(n: int) -> int:
    ergebnis = 1                    # 0! = 1! = 1
    for i in range(1, n + 1):
        ergebnis *= i
    return ergebnis

print(fakultaet_iterativ(5), fakultaet_iterativ(0))
```

Ausgabe

```
120 1
```

Beispiel: `fakultaet(-1)` mit der Abbruchbedingung `n == 0` endet in `RecursionError: maximum recursion depth exceeded`. Robuster wäre `if n <= 0`.

Rekursion: Vorteile	Rekursion: Nachteile
elegant und kurz bei selbstähnlichen Problemen (Fakultät, Fibonacci, Verzeichnisbäume, Sortieren)	speicherintensiv, denn jeder Aufruf braucht einen Rahmen (→ RecursionError)
mathematische Definitionen lassen sich direkt umsetzen	langsamer als Schleifen, weil es viele Aufrufe gibt
wenig Zustand und wenige Hilfsvariablen	schwerer zu debuggen; das „Rückwärtsdenken“ muss man lernen

Übungen Teil 2 (Lösungsansätze der Musterlösung)

```python
def maximum(zahlen: list[int], index: int = 0) ->
int:
    if index == len(zahlen) - 1:      # letztes
Element
        return zahlen[index]
    rest = maximum(zahlen, index + 1)
    return zahlen[index] if zahlen[index] > rest
else rest

def zaehle_zeichen(text: str, zeichen: str) -> int:
    if text == "":
        return 0
    treffer = 1 if text[0] == zeichen else 0
    return treffer + zaehle_zeichen(text[1:],
zeichen)
```

```python
def fibonacci(n: int) -> int:
    if n < 2:                  # fib(0) = 0, fib(1)
= 1
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

def potenz(basis: float, exponent: int) -> float:
    if exponent == 0:
        return 1.0
    if exponent < 0:
        return 1 / potenz(basis, -exponent)
    return basis * potenz(basis, exponent - 1)

print(fibonacci(10), potenz(2, 4), potenz(2, -2))
```

Ausgabe

```
55 16.0 0.25
```

```
print(maximum([3, 8, -2, 5]), maximum([-5, -2,
-9]))
print(zaehle_zeichen("banana", "a"))
```

Ausgabe

```
8 -2
3
```

Die naive Fibonacci-Rekursion ist sehr langsam, weil sie dieselben Werte immer wieder neu berechnet: `fibonacci(n)` ruft sich `2 · fibonacci(n + 1) − 1`-mal auf, bei n = 25 also 242 785-mal. `@functools.cache` über der Funktion merkt sich jedes Ergebnis, dann wird jedes `n` nur einmal berechnet. Die Lösung zu `maximum([])` deckt die leere Liste nicht ab, der Aufruf endet in einem RecursionError.

4.8 Funktionen testen: `assert` und pytest

`assert bedingung` tut nichts, wenn die Bedingung stimmt, und löst sonst einen `AssertionError` aus. **pytest** entspricht JUnit in Java. Die Testdatei heißt `test_….py` und liegt neben dem Code. Jede Testfunktion beginnt mit `test_` und enthält ein oder mehrere `assert`. Eingerichtet wird pytest einmal pro Projekt mit `uv add --dev pytest`, ausgeführt mit `uv run pytest`. In der Ausgabe steht `.` für einen bestandenen und `F` für einen fehlgeschlagenen Test.

```
# test_fakultaet.py
from fakultaet import fakultaet

def test_fakultaet_null() -> None:
    assert fakultaet(0) == 1

def test_fakultaet_fuenf() -> None:
    assert fakultaet(5) == 120

# Steht aus Versehen n + fakultaet(n - 1) im Code,
# meldet pytest: assert 16 == 120
#   where 16 = fakultaet(5)
```

Der Import `from fakultaet import fakultaet` ist nur ohne Nebenwirkungen möglich, weil `main()` hinter `if __name__ == "__main__":` steht.

Teste auch Randfälle, etwa `maximum([3])`, `maximum([-5, -2, -9]) == -2` und `zaehle_zeichen("", "a") == 0`. Floats prüfst du nicht mit `==`: `potenz(0.1, 3)` ergibt `0.0010000000000000002`. *(Ergänzung)*

4.9 `print` oder `return`?

Die häufigste Verwechslung in Klausuren: `print` **zeigt** einen Wert auf dem Bildschirm an, `return` **gibt** ihn an den Aufrufer zurück. Mit einem angezeigten Wert kann das Programm nicht weiterrechnen, mit einem zurückgegebenen schon. Bild im Kopf: `print` ist die Anzeige an der Kasse, die der Kunde sieht. `return` ist der Kassenbon, den der Kunde in die Hand bekommt und mitnimmt. *(Ergänzung, aufbauend auf 4.2)*

	`print(wert)`	`return wert`
Was passiert?	Text erscheint im Terminal	Wert geht an die Stelle des Aufrufs zurück
Funktion endet?	nein, es geht mit der nächsten Zeile weiter	**ja, sofort** (auch mitten in einer Schleife)
Was liefert der Aufruf?	`None` (wenn sonst kein `return` kommt)	den Wert
Weiterrechnen möglich?	nein: `None + 10` ist ein `TypeError`	ja: `f(100) + 10`
Rückgabetyp	`-> None`	z. B. `-> float`, `-> bool`, `-> str`
Signalwörter in der Aufgabe	„gibt … aus“, „zeigt an“, „druckt“	„gibt … zurück“, „liefert“, „berechnet“ (für die Weiterverarbeitung)

```
def brutto_print(netto: float) -> None:
    print(netto * 1.19)  # zeigt an, liefert None

def brutto_return(netto: float) -> float:
    return netto * 1.19  # liefert den Wert

a = brutto_print(100)
b = brutto_return(100)
print("a =", a)
print("b =", b)
print(brutto_return(100) + 10)
```

Ausgabe

```
119.0
a = None
b = 119.0
129.0
```

- `a = brutto_print(100)` zeigt `119.0` an, aber in `a` landet `None`.
- `b = brutto_return(100)` zeigt nichts an. Erst das `print("b =", b)` zeigt den Wert.
- Nur mit dem zurückgegebenen Wert lässt sich weiterrechnen: `brutto_return(100) + 10` ergibt 129.0.
- Mit dem `print`-Ergebnis geht das nicht: `brutto_print(100) + 10` zeigt erst 119.0 an und bricht dann ab mit `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'`.

Doppelte Ausgabe mit `None`: `print(f())` um eine Funktion, die selbst druckt, gibt zuerst deren Text aus und danach ihren Rückgabewert, also `None`.

```
def gruss(name: str) -> None:
    print("Hallo", name)

print(gruss("Ben"))
```

Ausgabe

```
Hallo Ben
None
```

`__str__` und `__repr__` müssen `return` verwenden. Mit `print` darin erscheint der Text zwar einmal, danach bricht `print(objekt)` ab:

```
class Konto:
    def __init__(self, inhaber: str) -> None:
        self.inhaber = inhaber

    def __str__(self) -> str:
        print(f"Konto von {self.inhaber}")  #
FEHLER: print statt return

print(Konto("Anna"))
```

Ausgabe

```
Konto von Anna
TypeError: __str__ returned non-string (type
NoneType)
```

Faustregeln

☐ Rechnen, prüfen, suchen → `return`. Ausgeben passiert möglichst an einer Stelle, meist in `main()`.

☐ Eine Funktion mit `-> None` darf drucken (z. B. `anzeigen()`), liefert aber nichts.

☐ Kaffeeshop (Kap. 7): `hinzufuegen()` liefert nur `True` / `False`, die Meldung schreibt `main.py`. So bleibt die Klasse ohne `input` und `print` testbar.

So erkennst du den Fehler

- In der Ausgabe steht ein unerwartetes `None` → irgendwo wird das Ergebnis einer `print`-Funktion ausgegeben.
- `'NoneType'` in einer Fehlermeldung → ein Wert, den du für eine Zahl oder ein Objekt hältst, ist `None`. Häufige Ursachen: `print` statt `return`, ein Zweig ohne `return`, `finden()` hat nichts gefunden.

☐ Tests mit `assert` /pytest (4.8) gehen nur mit `return`.

- `ty` meldet bei `-> float` ohne `return`: `invalid-return-type`.

Typische Fehler und Fallen

- `def` allein führt nichts aus. Ohne Klammern (`main` statt `main()`) wird die Funktion nicht aufgerufen.
- `print` statt `return`: Das Ergebnis ist `None`. Fehlt in einem Zweig das `return`, kommt dort ebenfalls `None` heraus.
- Eine lokale Variable außerhalb der Funktion benutzen gibt einen `NameError`. `zaehler += 1` auf eine globale Variable gibt einen `UnboundLocalError`.
- Neubindung (`liste = […]`) wirkt nur lokal, eine Veränderung (`liste[0] = …`, `append`) sieht auch der Aufrufer.
- Eine veränderbare Liste als Standardwert wird zwischen den Aufrufen geteilt, nimm `None`.
- Parameter mit Standardwert müssen hinter denen ohne stehen, sonst gibt es einen `SyntaxError`. Ein zweites `def` mit demselben Namen überschreibt das erste.
- Ist die Abbruchbedingung nicht erreichbar, z. B. bei `fakultaet(-1)` mit `n == 0`, gibt es einen `RecursionError`.

Das musst du können

☐ Funktionen mit Typannotationen, Docstring, `return` und `main()` samt `if __name__ == "__main__":` schreiben
☐ vorhersagen, was eine Funktion zurückgibt (auch `None`), und `break`, `continue` und `return` unterscheiden
☐ LEGB erklären und `NameError` und `UnboundLocalError` erkennen
☐ vorhersagen, ob eine Änderung im Aufrufer sichtbar ist (Neubindung oder Veränderung des Objekts)
☐ Standardwerte, Schlüsselwortargumente und `*args` verwenden und Javas Überladung in Python umsetzen
☐ rekursive Funktionen mit Abbruchbedingung schreiben (Fakultät, Summe, Fibonacci, Maximum, Zeichen zählen, Potenz) und den Aufrufstapel zeichnen
☐ imperativ und applikativ vergleichen; Tests mit `assert` und pytest schreiben
☐ `print` und `return` unterscheiden und vorhersagen, wo ein `None` auftaucht (4.9)

VL5 Objektorientierte Programmierung I

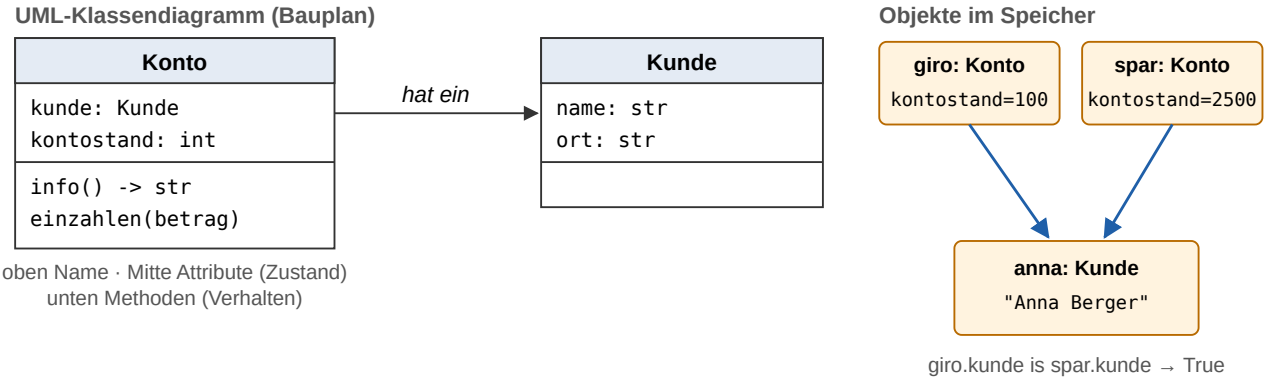
> **Hinweis**
> VL5 lief am **23.10.** (bis Übung 3) und am **27.10.** (Properties/Kapselung mit Übung 4, Hat-ein, Module, `@dataclass` mit Übung 5). Grundlage sind die Folien „vl05_oop_1“ und die Musterlösungen (VL05_Loesungen).

5.1 Klassen und Objekte

Bei der **objektorientierten Programmierung (OOP)** besteht ein Programm aus **Objekten**, die zusammenarbeiten. Ein Objekt hat **Attribute**, also Daten, die seinen Zustand beschreiben, und **Methoden**, also Funktionen, die sein Verhalten beschreiben und den Zustand ändern können. Jedes Objekt gehört zu einer **Klasse**, dem Bauplan, nach dem die Objekte gebaut werden. Aus einer Klasse kannst du beliebig viele Objekte erzeugen, ein Objekt heißt deshalb auch **Instanz** der Klasse. Jedes Objekt hat eigene Werte für die Attribute, die Methoden stehen aber nur einmal in der Klasse und gelten für alle Objekte.

Begriff	Bedeutung	Beispiel
Klasse	Bauplan, Name in PascalCase	`class Konto:`
Objekt / Instanz	ein konkretes Exemplar nach diesem Bauplan	`anna = Konto("Anna", 100)`
Attribut (Instanzattribut)	Daten bzw. Zustand eines Objekts, Zugriff mit dem Punkt	`anna.kontostand`
Methode	Funktion in der Klasse; ihr erster Parameter ist `self`	`anna.einzahlen(50)`
Konstruktor	`__init__`: läuft automatisch beim Erzeugen und setzt den Anfangszustand	`def __init__(self, …) -> None:`
`self`	das Objekt, für das die Methode läuft (Java: `this`)	`self.kontostand += betrag`
Klassenattribut	ein Wert, der der Klasse gehört und von allen Objekten geteilt wird (Java: `static`)	`Konto.anzahl_konten`

Objekte benutzt du schon lange: `42` ist ein Objekt der Klasse `int`, `"hallo"` ist ein `str`-Objekt mit Methoden wie `upper()`, und `noten.append(3)` verändert den Zustand eines `list`-Objekts. Auch `str(42)` und `list()` rufen Klassen auf und erzeugen damit Objekte.



Eine Klasse beschreibt, welche Attribute und Methoden ihre Objekte haben. Beide Konten zeigen auf dasselbe Kunden-Objekt (Hat-ein-Beziehung, Kap. 5.6).

5.2 Eine Klasse in Python: `__init__` und `self`

```
class Konto:
    def __init__(self, inhaber: str, kontostand: int) -> None:
        self.inhaber = inhaber
        self.kontostand = kontostand

def main() -> None:
    anna = Konto("Anna", 100)
    ben = Konto("Ben", 250)
    ben.kontostand = 300
    print(anna.inhaber, anna.kontostand)
```

Was bei `Konto("Anna", 100)` passiert:

1. Python legt ein neues, noch leeres Objekt an und ruft damit `__init__` auf. `self` zeigt auf dieses neue Objekt.
2. `self.inhaber = inhaber` legt das Attribut im Objekt an. Links steht das Attribut, rechts der Parameter, und beide dürfen gleich heißen.
3. Genauso entsteht `self.kontostand`. `__init__` gibt nichts zurück (`-> None`).

```
    print(ben.inhaber, ben.kontostand)

if __name__ == "__main__":
    main()
```

Ausgabe
```
Anna 100
Ben 300
```

4. Der Rahmen von `__init__` verschwindet, das Objekt bleibt. `Konto(…)` liefert es zurück, und `anna` zeigt jetzt darauf.

`ben.kontostand = 300` ändert nur das Objekt, auf das `ben` zeigt.

- Ein **`new` wie in Java gibt es nicht**, du rufst einfach die Klasse auf.
- Attribute werden **nicht vorab im Klassenrumpf deklariert** wie in Java. Sie entstehen in `__init__`, ihr Typ ergibt sich aus dem annotierten Parameter.
- **Genau ein `__init__`**, denn Überladung gibt es nicht. Varianten baust du mit Standardwerten: Mit `kontostand: int = 0` funktionieren `Konto("Anna", 100)` und `Konto("Ben")`. Ohne `__init__` erzeugt `Konto()` ein Objekt ohne Attribute.
- Namen mit doppelten Unterstrichen wie `__init__` haben für Python eine feste Bedeutung. Man nennt sie **Dunder-Methoden** („double underscore“).
- Beträge stehen auf den Folien als `int` in ganzen Euro, damit der Code kurz bleibt. Für Cent-Beträge nimmt man `Decimal`.

5.3 Methoden, `self`, `__str__` und `__repr__`

```
class Konto:
    def __init__(self, inhaber: str, kontostand: int = 0) ->
None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    def einzahlen(self, betrag: int) -> None:
        self.kontostand += betrag

    def abheben(self, betrag: int) -> bool:
        if betrag > self.kontostand:
            print("Abgelehnt: Der Kontostand reicht nicht.")
            return False
        self.kontostand -= betrag
        return True

anna = Konto("Anna", 100)
ben = Konto("Ben", 250)
anna.einzahlen(50)
ben.einzahlen(20)
print(anna.kontostand, ben.kontostand)
print(anna.abheben(500))
print(anna.abheben(30), anna.kontostand)
Konto.einzahlen(anna, 10)   # dasselbe wie anna.einzahlen(10)
print(anna.kontostand)
```

Ausgabe
```
150 270
Abgelehnt: Der Kontostand reicht nicht.
False
True 120
130
```

Eine **Methode** ist eine Funktion, die in der Klasse steht. Ihr erster Parameter ist `self`. Über `self` liest und ändert die Methode die Attribute ihres Objekts. Die Regel „nur abheben, wenn genug Geld da ist“ steht damit an einer einzigen Stelle, nämlich in der Klasse.

Woher kommt `self`? Bei `anna.einzahlen(50)` sucht Python die Methode in der Klasse von `anna` und ruft `Konto.einzahlen(anna, 50)` auf. Das Objekt vor dem Punkt wird so zu `self` und die 50 zu `betrag`. Deshalb hat `einzahlen` zwei Parameter, beim Aufruf steht aber nur ein Argument in den Klammern.

`self` entspricht `this` in Java. In Python steht es aber ausdrücklich in jeder Methode, und vor Attribute schreibt man immer `self.`.

Bei `ben.einzahlen(20)` zeigt `self` auf Bens Objekt. Es ist dieselbe Methode, aber ein anderes `self`.

```
class Konto:
    def __init__(self, inhaber: str, kontostand: int = 0) ->
None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    def __str__(self) -> str:        # für Menschen: print()
        return f"Konto von {self.inhaber}: {self.kontostand}
Euro"

    def __repr__(self) -> str:       # für Entwickler: REPL,
Listen
        return f"Konto({self.inhaber!r}, {self.kontostand})"
```

Methode	wofür	wird genutzt von
`__str__`	lesbarer Text für Menschen (Java: `toString()`)	`print(obj)`, `str(obj)`, f-Strings
`__repr__`	sieht möglichst aus wie der Aufruf, der das Objekt erzeugt; `!r` setzt die Anführungszeichen	REPL, Ausgabe von Listen, `repr(obj)`

Ohne diese Methoden gibt `print(anna)` nur etwas wie `<konto.Konto object at 0x10349f770>` aus.

```
anna = Konto("Anna", 100)
print(anna)
print(repr(anna))
print([anna, Konto("Ben")])
```

Ausgabe

```
Konto von Anna: 100 Euro
Konto('Anna', 100)
[Konto('Anna', 100), Konto('Ben', 0)]
```

5.4 Namen zeigen auf Objekte, auch bei eigenen Klassen

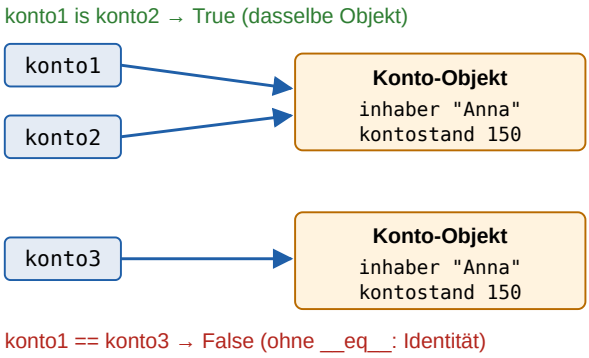
```
class Konto:
    def __init__(self, inhaber: str, kontostand: int =
0) -> None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    def einzahlen(self, betrag: int) -> None:
        self.kontostand += betrag

konto1 = Konto("Anna", 100)
konto2 = konto1            # KEIN zweites Konto!
konto2.einzahlen(50)
konto3 = Konto("Anna", 150)
print(konto1.kontostand)
print(konto1 is konto2, konto1 == konto3)
```

Ausgabe

```
150
True False
```



- `konto2 = konto1` kopiert nichts. Es hängt nur ein zweites Namensschild an dasselbe Objekt, so wie bei Listen in VL3 und VL4. Eine Änderung über `konto2` siehst du deshalb auch über `konto1`.
- `==` vergleicht Objekte einer eigenen Klasse **ohne `__eq__`** nur auf **Identität**, also genauso wie `is`. Mit `@dataclass` ändert sich das (Kap. 5.7).
- Für „noch kein Objekt“ gibt es `None` (Java: `null`). Geprüft wird mit `if konto is None:`.

Attribute sind dynamisch, Tippfehler auch

```
anna = Konto("Anna", 100)
anna.kontostnad = 500   # Tippfehler: kein Fehler,
                        # sondern ein NEUES
Attribut!
print(anna.kontostand)  # 100

$ uv run ty check
error[unresolved-attribute]: Unresolved attribute
`kontostnad` on type `Konto`
```

Python erlaubt, einem Objekt jederzeit neue Attribute zu geben, deshalb gibt es zur Laufzeit **keinen Fehler**. `ty` kennt aber die Attribute aus `__init__` und meldet den Tippfehler schon vor der Ausführung. **Deshalb legst du alle Attribute in `__init__` an**, auch die, die am Anfang leer sind (`None`, `[]`).

5.5 Klassenattribute

```
class Konto:
    anzahl_konten = 0        # Klassenattribut: einmal, in der
Klasse

    def __init__(self, inhaber: str, kontostand: int = 0) ->
None:
        Konto.anzahl_konten += 1     # immer über den
Klassennamen
        self.nummer = f"K{Konto.anzahl_konten:03}"
        self.inhaber = inhaber       # Instanzattribute
        self.kontostand = kontostand

anna = Konto("Anna", 100)
ben = Konto("Ben")
print(anna.nummer, ben.nummer, Konto.anzahl_konten)
ben.anzahl_konten += 1       # FALLE
print(ben.anzahl_konten, Konto.anzahl_konten,
anna.anzahl_konten)
```

Ausgabe

```
K001 K002 2
3 2 2
```

Ein **Klassenattribut** steht direkt im Klassenrumpf. Es gibt den Wert nur einmal, nämlich in der Klasse, und alle Objekte teilen ihn. Hier zählt es die Konten und liefert die nächste Kontonummer (`:03` füllt auf drei Stellen mit Nullen auf). Klassenattribute entsprechen den `static`-Variablen in Java.

Falle: Über ein Objekt kannst du ein Klassenattribut *lesen*, denn `ben.anzahl_konten` findet das Klassenattribut, wenn das Objekt selbst keins mit diesem Namen hat. *Zuweisen* über ein Objekt (`ben.anzahl_konten += 1`) legt dagegen ein neues **Instanzattribut** an, das nur `ben` hat. Der Zähler in der Klasse bleibt

bei 2. Klassenattribute änderst du deshalb **immer über den Klassennamen**.

5.6 Kapselung mit Properties

OOP-Prinzip	Bedeutung	wann
Kapselung	Daten und Methoden gehören zusammen. Das Objekt bestimmt selbst, wie sein Zustand geändert wird, und zeigt nach außen nur das Nötige.	VL5
Vererbung	Eine Klasse übernimmt Attribute und Methoden einer anderen („ist ein“: Ein Girokonto ist ein Konto).	VL6/7
Polymorphismus	Objekte verschiedener Klassen reagieren unterschiedlich auf denselben Methodenaufruf.	VL6/7
Abstraktion	Gemeinsamkeiten mehrerer Klassen werden in einer Oberklasse zusammengefasst, unwichtige Details weggelassen.	VL6/7

Warum Kapselung? Mit der Klasse aus 5.2 kann jeder Teil des Programms den Kontostand beliebig setzen, auch auf einen ungültigen Wert wie `anna.kontostand = -999`. Bei Kapselung schützt das Objekt seinen inneren Zustand: Die Klasse prüft selbst, ob eine Änderung erlaubt ist, und das Objekt bleibt immer in einem gültigen Zustand. Außerdem kannst du das Innere ändern, ohne anderen Code anpassen zu müssen.

Java	Python	Bedeutung in Python
`public`	`kontostand`	Standard: Alles ohne Unterstrich ist öffentlich.
`private`	`_kontostand`	Konvention: intern, nur innerhalb der Klasse benutzen
`protected`	`_kontostand`	wie oben, Python unterscheidet die beiden Fälle nicht
package-private	–	gibt es nicht

Java prüft die Sichtbarkeit beim Übersetzen. Python prüft den Unterstrich nicht: `anna._kontostand` funktioniert, aber wer es benutzt, verlässt sich auf Interna der Klasse. Bei **zwei** Unterstrichen (`__kontostand`) benennt Python das Attribut intern in `_Konto__kontostand` um. Das verhindert Namenskonflikte bei Vererbung, schützt den Wert aber nicht. **Getter und Setter** schreibt man in Java für fast jedes Attribut, in Python ist das unüblich. Solange es nichts zu prüfen gibt, bleibt das Attribut öffentlich, erst wenn es Regeln gibt, kommt eine **Property** dazu.

```
class Mitarbeiter:
    def __init__(self, name: str, monatsgehalt: int) -> None:
        self.name = name
        self.monatsgehalt = monatsgehalt

    @property
    def jahresgehalt(self) -> int:      # berechnet, nur lesbar
        return self.monatsgehalt * 12

lena = Mitarbeiter("Lena", 4200)
print(lena.jahresgehalt)                # ohne Klammern!
lena.monatsgehalt = 4500
print(lena.jahresgehalt)
lena.jahresgehalt = 60000
```

Ausgabe

```
50400
54000
Traceback … (gekürzt)
AttributeError: property 'jahresgehalt' of 'Mitarbeiter' object
has no setter
```

Eine **Property** ist eine Methode, die man von außen wie ein Attribut benutzt, also ohne Klammern: `lena.jahresgehalt`. Ihr Wert wird bei jedem Zugriff neu berechnet und passt deshalb immer zum Monatsgehalt.

Ohne Setter ist eine Property **nur lesbar**. `ty` meldet `Cannot assign to read-only property`, zur Laufzeit gibt es einen `AttributeError`.

```
from decimal import Decimal

class Konto:
    def __init__(self, inhaber: str, kontostand: Decimal) ->
None:
        self.inhaber = inhaber
        self._kontostand = Decimal(0)
        self.kontostand = kontostand   # nutzt den Setter

    @property
    def kontostand(self) -> Decimal:   # Getter
        return self._kontostand

    @kontostand.setter
    def kontostand(self, neuer_wert: Decimal) -> None:
        if neuer_wert < 0:
            print("Ungültig: negativer Kontostand.")
```

Der Wert liegt im internen Attribut `_kontostand`, die Property `kontostand` ist die öffentliche Schnittstelle.
`anna.kontostand` ruft den **Getter** auf, `anna.kontostand = x` den **Setter**.

Auch der Konstruktor benutzt den Setter, damit die Regel schon beim Anlegen gilt.

Laut Folie wäre bei ungültigen Werten eigentlich eine Ausnahme üblich (`raise ValueError(…)`) statt `print`.

Situation	In Python	Beispiel

```
            return
        self._kontostand = neuer_wert

    def einzahlen(self, betrag: Decimal) -> None:
        self.kontostand += betrag

anna = Konto("Anna", Decimal("100.00"))
anna.einzahlen(Decimal("49.99"))
print(repr(anna.kontostand))
anna.kontostand = Decimal("-500")
print(anna.kontostand)
```

Ausgabe

```
Decimal('149.99')
Ungültig: negativer Kontostand.
149.99
```

Wert darf frei geändert werden	öffentliches Attribut	`self.inhaber`
von außen nur lesen	`_attribut` + Property ohne Setter	Bestand, der sich nur über Methoden ändert
Änderung nur unter Regeln	`_attribut` + Property mit Setter	`kontostand`
Wert wird berechnet	Property ohne eigenes Attribut	`jahresgehalt`

Von außen sieht eine Property aus wie ein Attribut. Du kannst also mit einem öffentlichen Attribut anfangen und später eine Property daraus machen, ohne dass sich der aufrufende Code ändert. Hilfsmethoden, die nur in der Klasse gebraucht werden, bekommen ebenfalls einen Unterstrich: `_pruefe_betrag()`.

Java-Modifikator	Zweck	Python
`static` Variable	Zähler, geteilte Werte	Klassenattribut (`anzahl_konten = 0`)
`static final`	Konstanten	Modulkonstante `MAX_KONTEN: Final = 100`
`static` Methode	Hilfsmethoden	meist eine normale Funktion im Modul; `@staticmethod` gibt es auch
`final` Methode/Klasse	Überschreiben verhindern	`@final` aus `typing`, prüft nur ty (VL6)
`abstract`	unvollständige Klassen	`abc.ABC`, `@abstractmethod` (VL6/07)

5.7 Zusammengesetzte Klassen, Module und `@dataclass`

Objekte können andere Objekte als Attribute haben. Das nennt man eine **Hat-ein-Beziehung** (has-a): Ein Konto hat einen Kunden, eine Bestellung hat einen Kunden und mehrere Positionen, eine Abteilung hat mehrere Mitarbeiter. Damit bildest du Zusammenhänge aus der realen Welt direkt ab. Jede Klasse bleibt klein und kümmert sich um ihre eigene Aufgabe, und die Objekte arbeiten zusammen, indem sie gegenseitig ihre Methoden aufrufen.

```
class Kunde:
    def __init__(self, name: str, ort: str) -> None:
        self.name = name
        self.ort = ort

class Konto:
    def __init__(self, kunde: Kunde, kontostand: int = 0) ->
None:
        self.kunde = kunde          # Konto HAT einen Kunden
        self.kontostand = kontostand

    def info(self) -> str:
        return f"{self.kunde.name}: {self.kontostand} Euro"

anna = Kunde("Anna Berger", "Heilbronn")
giro = Konto(anna, 100)
spar = Konto(anna, 2500)
print(spar.info())
print(giro.kunde is spar.kunde)
```

Ausgabe

```
Anna Berger: 2500 Euro
True
```

Der Parameter `kunde` hat als Typ die eigene Klasse `Kunde`. Über das Attribut kommt das Konto an die Daten des Kunden heran: `self.kunde.name`. Beide Konten zeigen auf **dasselbe** Kunden-Objekt (siehe Diagramm in 5.1).

Module und Pakete: Jede `.py`-Datei ist ein **Modul**. Ein Ordner mit Modulen und einer Datei `__init__.py` (die leer sein darf) ist ein **Paket**. Pakete ordnen große Projekte und vermeiden Namenskonflikte, denn `banking.konto` und `lager.konto` dürfen beide existieren. Der Paketname ergibt sich aus dem Ordner, einen `package`-Kopf wie in Java gibt es nicht. Mehrere zusammengehörige Klassen dürfen in einem Modul stehen.

```
bank/
├── main.py      # from banking.konto
import Konto
└── banking/     # from banking.kunde
import Kunde
    ├── __init__.py
    ├── konto.py
    └── kunde.py
```

```
from dataclasses import dataclass

@dataclass
class Kunde:
    name: str
    ort: str

k = Kunde("Anna Berger", "Heilbronn")
print(k)
print(k == Kunde("Anna Berger", "Heilbronn"))
```

Ausgabe

```
Kunde(name='Anna Berger', ort='Heilbronn')
True
```

`@dataclass` ist eine Kurzform für Datenklassen. Aus den annotierten Attributen im Klassenrumpf erzeugt Python `__init__`, `__repr__` und `__eq__` automatisch. `==` vergleicht dann die **Werte** und nicht mehr die Identität. Von Hand müsstest du dafür `__eq__` mit `isinstance` und `NotImplemented` selbst schreiben. Auch eine Datenklasse darf Methoden und Properties haben. `@dataclass` nimmt dir nur Schreibarbeit ab, heraus kommt dieselbe Klasse, die du sonst von Hand schreiben würdest.

	`@dataclass`	normale Klasse
Zweck	vor allem Daten bündeln	Verhalten und Regeln stehen im Vordergrund
Beispiele	`Kunde`, `Adresse`, `Bestellposition`	`Konto`, `Warenkorb`, `Lager`
`==`	vergleicht die Werte	vergleicht die Identität (ohne eigenes `__eq__`)
Schreibarbeit	gering	`__init__` usw. von Hand

Übung 5: Warenkorb (vereinfacht nach der Musterlösung; dort ist `Kunde` eine `@dataclass` mit `name` und `email`)

```
from dataclasses import dataclass
from decimal import Decimal

@dataclass
class Produkt:            # in der Lösung: normale Klasse mit Properties (Übung 4)
    name: str
    preis: Decimal
    bestand: int

@dataclass
class Position:           # eine Position HAT ein Produkt
    produkt: Produkt
    menge: int

    def summe(self) -> Decimal:
        return self.produkt.preis * self.menge

class Warenkorb:          # HAT einen Kunden und mehrere Positionen
    def __init__(self, kunde: str) -> None:
        self.kunde = kunde
        self.positionen: list[Position] = []   # leere Liste in __init__

    def hinzufuegen(self, produkt: Produkt, menge: int) -> None:
        if menge > produkt.bestand:
            print(f"Nur noch {produkt.bestand} x {produkt.name} auf Lager.")
            return
        self.positionen.append(Position(produkt, menge))

    def gesamtsumme(self) -> Decimal:
        summe = Decimal(0)
        for position in self.positionen:
            summe += position.summe()
        return summe

korb = Warenkorb("Anna Berger")
kaffee = Produkt("Kaffee 1 kg", Decimal("12.99"), 40)
korb.hinzufuegen(kaffee, 2)
korb.hinzufuegen(Produkt("Grüner Tee", Decimal("3.49"), 120), 3)
korb.hinzufuegen(kaffee, 50)
print(f"Summe: {korb.gesamtsumme()} Euro")
```

Ausgabe

```
Nur noch 40 x Kaffee 1 kg auf Lager.
Summe: 36.45 Euro
```

Weitere Übungen aus der Musterlösung: In Übung 4 bekommt `Produkt` den Bruttopreis als Property: `(self._preis * (1 + MWST_SATZ)).quantize(Decimal("0.01"))` ergibt bei 12,99 € den Wert 15.46. `kaffee.bestand = 100` scheitert dort, weil `bestand` eine Property ohne Setter ist (AttributeError). In Übung 3 ergibt `kaffee.anzahl += 1` ein Instanzattribut, `Produkt.anzahl` bleibt unverändert.

OOP: Vor- und Nachteile, Geschichte

Vorteile	Nachteile
Struktur: Daten und Verhalten an einem Ort	**Einstieg**: Vererbung, Polymorphismus und Referenzen brauchen Übung
Wiederverwendung: Klassen lassen sich erweitern und kombinieren	**Aufwand**: Objekte brauchen mehr Speicher und Laufzeit
Kapselung: klar definierte Schnittstellen	für kleine Aufgaben oft zu umständlich
Abstraktion: Fokus auf das Wesentliche; **Erweiterbarkeit**: neue Klassen fügen sich leicht ein	**Overengineering**: zu viele Klassen für ein kleines Problem

Geschichte: In den 1980ern stößt die prozedurale Programmierung bei großen Systemen an ihre Grenzen (wenig Modularität, schwer wartbar). **Simula** (1967) und **Smalltalk** (1970er) legen das Fundament mit Objekten und Nachrichten, **C++** (1980er) bringt OOP in die Industrie, **Java** (1995) verbreitet sie weiter. **Python** (1991) ist von Anfang an objektorientiert gebaut: Jeder Wert ist ein Objekt, eigene Klassen sind trotzdem freiwillig. Für kleine Aufgaben reichen oft Funktionen.

Typische Fehler und Fallen

- `self` in der Methodendefinition vergessen: Beim Aufruf gibt es dann einen `TypeError`, weil Python das Objekt trotzdem als erstes Argument übergibt. Ebenso falsch ist, `self.` vor einem Attribut zu vergessen (`kontostand += betrag` statt `self.kontostand += betrag`).
- `__init__` gibt nichts zurück (`-> None`), das Objekt liefert der Aufruf `Konto(…)`. Ein `new` gibt es nicht.
- `konto2 = konto1` erzeugt kein zweites Objekt.
- `==` ohne `__eq__` bzw. ohne `@dataclass` vergleicht die Identität. Zwei Objekte mit gleichen Werten sind dann nicht gleich.
- Ein Tippfehler bei der Attributzuweisung legt still ein neues Attribut an, nur `ty` findet ihn.
- Ein Klassenattribut über ein Objekt ändern legt ein Instanzattribut an. Ändere es immer über den Klassennamen.
- Eine Property ruft man ohne Klammern auf (`lena.jahresgehalt`). Ohne Setter ist sie nur lesbar (AttributeError).
- `_attribut` ist nur eine Konvention, Python schützt nichts.
- Ohne `__str__` zeigt `print(obj)` nur `<… object at 0x…>`.

Das musst du können

☐ Klasse, Objekt/Instanz, Attribut, Methode, Konstruktor, `self` und Klassenattribut erklären und an Code zeigen
☐ eine Klasse aus einem UML-Klassendiagramm programmieren (mit `__init__`, Typannotationen und Methoden mit Prüfregeln)
☐ erklären, warum `anna.einzahlen(50)` dasselbe ist wie `Konto.einzahlen(anna, 50)`
☐ `__str__` und `__repr__` schreiben und ihren Unterschied nennen
☐ `is` und `==` bei eigenen Objekten vorhersagen, mit und ohne `@dataclass`
☐ einen Zähler als Klassenattribut programmieren und die Zuweisungsfalle über das Objekt erklären
☐ Kapselung begründen; `_attribut`, eine Property ohne und mit Setter sowie eine berechnete Property schreiben
☐ die vier OOP-Prinzipien nennen; Hat-ein-Beziehungen, Module und Pakete erklären

VL6 OOP II: Vererbung und Polymorphie

Stand

VL6 (Folien „vl06_oop_2“, 60 Folien) begann am **03.11.** mit Vererbung, `super()` und Überschreiben (F. 1–29, Übung 1 und 2). Der Rest ab F. 30 (Polymorphie, `object`, abstrakte Klassen, Fähigkeiten, Übung 3 bis 5) war laut Plan am **06.11.** dran; bestätigt ist das nicht. Seit 09.11. liegt in Moodle eine Wiederholung VL01–VL06 am Beispiel Fitnessstudio (6.7). Die Beispiele folgen den Folien und Musterlösungen, Zahlen sind nachgerechnet.

6.1 Vererbung: „ist ein“ und `super().__init__`

Bei der **Vererbung** übernimmt eine Klasse alle Attribute und Methoden einer anderen und ergänzt nur, was neu ist (F. 6). Die allgemeinere Klasse heißt **Oberklasse** (Basisklasse), die speziellere **Unterklasse**. Die Oberklasse steht in Klammern hinter dem Klassennamen, das entspricht `extends` in Java (F. 9). Bild im Kopf: Ein Rezept „Zimtschnecken: wie Hefeteig, zusätzlich Zimt und Zucker“. Alles, was für Hefeteig gilt, gilt auch für Zimtschnecken.

```
class Konto:
    def __init__(self, inhaber: str, kontostand: int = 0) ->
None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    def einzahlen(self, betrag: int) -> None:
        self.kontostand += betrag

    def __str__(self) -> str:
        return f"Konto von {self.inhaber}: {self.kontostand}
Euro"

class Sparkonto(Konto):  # Sparkonto IST EIN Konto
    def __init__(self, inhaber: str, kontostand: int, zinssatz:
int) -> None:
        super().__init__(inhaber, kontostand)  # 1. Konto-Teil
anlegen
        self.zinssatz = zinssatz  # 2. das Neue

    def zinsen_gutschreiben(self) -> None:  # nur im Sparkonto
        self.einzahlen(self.kontostand * self.zinssatz // 100)

anna = Sparkonto("Anna", 1000, 2)
anna.einzahlen(500)  # geerbt
anna.zinsen_gutschreiben()  # 1500 * 2 // 100 = 30
print(anna)  # __str__ geerbt
print(vars(anna))
print(isinstance(anna, Konto), type(anna) is Konto,
issubclass(Sparkonto, Konto))
```

Ausgabe

```
Konto von Anna: 1530 Euro
{'inhaber': 'Anna', 'kontostand': 1530, 'zinssatz': 2}
True False True
```

Was bei `Sparkonto("Anna", 1000, 2)` passiert:

1. Python legt ein leeres Sparkonto-Objekt an und ruft `Sparkonto.__init__` auf.
2. `super().__init__(inhaber, kontostand)` ruft das `__init__` von `Konto` mit **demselben** `self` auf und legt `inhaber` und `kontostand` an.
3. Zurück in `Sparkonto.__init__` kommt `zinssatz` dazu. Ergebnis: ein Objekt mit allen drei Attributen.

Merksatz: „**Mach erst, was die Elternklasse beim Anlegen macht**, dann meinen Teil.“

Zeile	läuft aus …	Stand
`einzahlen(500)`	Konto (geerbt)	1500
`zinsen_…()`	Sparkonto: 1500 · 2 // 100 = 30	1530
`print(anna)`	`__str__` aus Konto	1530

Der Test: „ist ein“ → Vererbung, „hat ein“ → Attribut (F. 7). Bilde für jedes Paar beide Sätze und prüf, welcher stimmt.

Paar	Satz, der stimmt	also
Sparkonto – Konto	Ein Sparkonto **ist ein** Konto.	Vererbung
Werkstudent – Mitarbeiter	Ein Werkstudent **ist ein** Mitarbeiter.	Vererbung
Konto – Kunde	Ein Konto **hat einen** Kunden.	Attribut
Bestellung – Position	Eine Bestellung **hat** Positionen.	Attribut (Liste)

`isinstance(objekt, Klasse)` prüft die Klasse **und** alle Oberklassen (Java: `instanceof`), `type(objekt) is Klasse` nur die genaue Klasse, `issubclass(Unter,`

Falle: `super().__init__` vergessen. Python ruft das `__init__` der Oberklasse **nicht** von selbst auf (anders als Java, F. 10). Das Erzeugen klappt noch, der Fehler kommt erst bei der ersten Methode, die auf `kontostand` zugreift:

```
class Sparkonto(Konto):
    def __init__(self, inhaber: str, kontostand:
int, zinssatz: int) -> None:
        self.zinssatz = zinssatz  #
super().__init__ fehlt!

anna = Sparkonto("Anna", 1000, 2)  # läuft noch
ohne Fehler
anna.einzahlen(500)
```

Ausgabe

```
AttributeError: 'Sparkonto' object has no attribute
'kontostand'
```

(`Konto` wie oben.) Hat die Unterklasse **kein eigenes** `__init__`, wird das der Oberklasse einfach geerbt, dann ist nichts zu tun. `super.__init__(…)` ohne Klammern hinter

`Ober)` zwei Klassen (F. 15). Siehe letzte Zeile oben: `True False True`.

`super` ist falsch (`TypeError: descriptor '__init__' requires a 'super' object …`).

6.2 Überschreiben, `super().methode()`, `@override` und die Methodensuche

Überschreiben heißt: Die Unterklasse schreibt eine Methode mit **gleichem Namen und gleichen Parametern** neu. Für ihre Objekte gilt dann die neue Version (F. 19). Mit `super().methode()` benutzt sie die alte Version mit, sie **ergänzt** also statt zu **ersetzen** (F. 20). Beispiel aus VL6 Übung 1 und 2:

```
from typing import Final, override

FUEHRUNGSZULAGE: Final = 150

class Mitarbeiter:
    def __init__(self, name: str, monatsgehalt: int) -> None:
        self.name = name
        self.monatsgehalt = monatsgehalt

    def arbeiten(self) -> str:
        return f"{self.name} arbeitet."

    def monatskosten(self) -> int:
        return self.monatsgehalt

    def __str__(self) -> str:
        return f"{self.name}: {self.monatsgehalt} Euro im Monat"

class Entwickler(Mitarbeiter):
    def __init__(self, name: str, monatsgehalt: int, sprache:
str) -> None:
        super().__init__(name, monatsgehalt)
        self.sprache = sprache

    @override
    def arbeiten(self) -> str:  # ersetzen
        return f"{self.name} schreibt Code in {self.sprache}."

    @override
    def __str__(self) -> str:  # ergänzen
        return f"{super().__str__()}, {self.sprache}"

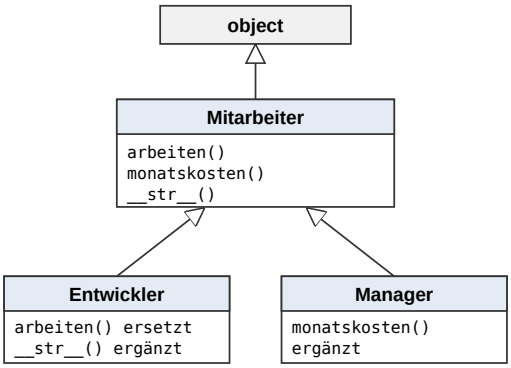
class Manager(Mitarbeiter):
    def __init__(self, name: str, monatsgehalt: int,
teamgroesse: int) -> None:
        super().__init__(name, monatsgehalt)
        self.teamgroesse = teamgroesse

    @override
    def monatskosten(self) -> int:  # ergänzen
        return super().monatskosten() + FUEHRUNGSZULAGE *
self.teamgroesse

lena = Entwickler("Lena", 4200, "Python")
tom = Manager("Tom", 5800, 6)
print(lena)
print(tom)
print(lena.arbeiten(), "|", tom.arbeiten())
print(tom.monatskosten())
print(isinstance(tom, Mitarbeiter), isinstance(tom, Entwickler))
print([k.__name__ for k in Manager.__mro__])
```

Ausgabe

```
Lena: 4200 Euro im Monat, Python
Tom: 5800 Euro im Monat
Lena schreibt Code in Python. | Tom arbeitet.
6700
True False
['Manager', 'Mitarbeiter', 'object']
```



tom.arbeiten(): 1. Manager? nein
2. Mitarbeiter? ja → „Tom arbeitet.“

UML: Der Pfeil mit offenem Dreieck zeigt zur Oberklasse („ist ein“).

Wo sucht Python eine Methode? Zuerst in der Klasse des Objekts, dann in der Oberklasse und so weiter bis `object`. Die **erste Fundstelle gewinnt** (F. 21–25). Die Reihenfolge steht in `Klasse.__mro__` (Method Resolution Order). `tom.monatskosten()` findet Python schon in `Manager`: 5800 + 150 · 6 = 6700.

`@override` aus `typing` sagt: „Diese Methode soll eine geerbte ersetzen.“ Python ignoriert das beim Ausführen, `ty` prüft es: Bei einem Tippfehler (`def arbeitn`) meldet es `invalid-explicit-override`. Ohne `@override` entstünde still eine neue Methode (F. 26). Import vergessen: `NameError: name 'override' is not defined`.

(F. 28)	Überladen	Überschreiben
wo?	in derselben Klasse	in Ober- und Unterklasse
Parameter	verschieden	gleich
in Python	gibt es nicht, ein zweites `def` ersetzt das erste (Kap. 4.5: Standardwerte statt Überladung)	ja, welche Version läuft, entscheidet die Klasse des Objekts

6.3 Polymorphie, Typangabe und Duck Typing

Polymorphie („Vielgestaltigkeit“): **Derselbe Methodenaufruf** führt je nach Klasse des Objekts zu **unterschiedlichem Verhalten** (F. 32). Einstiegsfrage der Folie: „Sie klicken im Onlineshop auf ‚Jetzt bezahlen‘. Was muss der Shop je nach Zahlungsart tun?“ Neu ist dabei nichts: Polymorphie ist Überschreiben plus eine Funktion oder Liste, die nur die Oberklasse kennt. Bild im Kopf: eine Universalfernbedienung mit einer Taste „Ein/Aus“. Was passiert, entscheidet das Gerät, auf das sie zeigt.

```
from decimal import Decimal
from typing import override

class Zahlungsart:
    def bezahlen(self, betrag: Decimal) -> str:
        return f"{betrag} Euro bezahlt"

class Kreditkarte(Zahlungsart):
    def __init__(self, nummer: str) -> None:
        self.nummer = nummer

    @override
    def bezahlen(self, betrag: Decimal) -> str:
        return f"{betrag} Euro von Karte ...{self.nummer[-4:]}
abgebucht"

class Rechnungskauf(Zahlungsart):
    @override
    def bezahlen(self, betrag: Decimal) -> str:
        return f"Rechnung über {betrag} Euro, zahlbar in 14
Tagen"

class Lastschrift(Zahlungsart):  # später dazugekommen
    def __init__(self, iban: str) -> None:
        self.iban = iban

    @override
    def bezahlen(self, betrag: Decimal) -> str:
        return f"{betrag} Euro von IBAN ...{self.iban[-4:]}
eingezogen"

def kasse(zahlungsart: Zahlungsart, betrag: Decimal) -> None:
    print(zahlungsart.bezahlen(betrag))  # immer dieselbe Zeile

kasse(Kreditkarte("4111111111111234"), Decimal("49.90"))
kasse(Rechnungskauf(), Decimal("199.00"))
kasse(Lastschrift("DE89370400440532013000"), Decimal("20.00"))
```

Ausgabe

```
49.90 Euro von Karte ...1234 abgebucht
Rechnung über 199.00 Euro, zahlbar in 14 Tagen
20.00 Euro von IBAN ...3000 eingezogen
```

Aufruf	`zahlungsart` zeigt auf	Python findet `bezahlen` in …
1	Kreditkarte-Objekt	`Kreditkarte`
2	Rechnungskauf-Objekt	`Rechnungskauf`
3	Lastschrift-Objekt	`Lastschrift`

Nutzen (F. 35): Für die Lastschrift kam nur eine neue Klasse dazu, `kasse()` blieb **unverändert**.

Typangabe oder Objekt: wer entscheidet was? (F. 34)

- Die **Typangabe** (`zahlungsart: Zahlungsart`) entscheidet, was **ty erlaubt**: nur, was *jede* Zahlungsart hat. `zahlungsart.bezahlen(…)` ist ok, `zahlungsart.nummer` meldet ty als `unresolved-attribute` (ein Rechnungskauf hat keine Nummer).
- Das **Objekt** entscheidet beim Ausführen, **welches** `bezahlen()` läuft.
- Braucht man die Nummer doch: vorher `if isinstance(zahlungsart, Kreditkarte):`. Im Block erlaubt ty dann `zahlungsart.nummer`. Einen Cast wie in Java braucht Python nicht.

Duck Typing (F. 39/40): Python prüft beim Ausführen nur, **ob die Methode da ist**, nicht, von welcher Klasse das Objekt stammt („Wenn es aussieht wie eine Ente und quakt wie eine Ente …“). Ein `Gutschein`, der **nicht** erbt, läuft deshalb. `ty` meldet aber `invalid-argument-type`, weil die Typangabe `Zahlungsart` verlangt. In der Vorlesung gilt: **immer erben**, dann prüft ty mit und jeder sieht die Absicht.

```
class Gutschein:  # erbt NICHT von Zahlungsart
    def bezahlen(self, betrag: Decimal) -> str:
        return f"{betrag} Euro mit Gutschein
bezahlt"

def kasse(zahlungsart: Zahlungsart, betrag:
Decimal) -> None:
    print(zahlungsart.bezahlen(betrag))

kasse(Gutschein(), Decimal("15.00"))  # läuft; ty:
invalid-argument-type
```

Ausgabe

```
15.00 Euro mit Gutschein bezahlt
```

VL6 Übung 3: Personalkosten (Klassen wie in 6.2). `personalkosten()` kennt nur `Mitarbeiter`, jedes Objekt rechnet seine Kosten selbst:

```
class Werkstudent(Mitarbeiter):
    def __init__(self, name: str, monatsgehalt:
int, stunden: int) -> None:
        super().__init__(name, monatsgehalt)
        self.stunden = stunden

    @override
    def arbeiten(self) -> str:
        return f"{self.name} arbeitet
{self.stunden} Stunden pro Woche mit."

def personalkosten(team: list[Mitarbeiter]) -> int:
    summe = 0  # Startwert VOR der Schleife
    for person in team:
        summe += person.monatskosten()  # jedes
Objekt rechnet selbst
    return summe  # NACH der Schleife

team: list[Mitarbeiter] = [
    Entwickler("Lena", 4200, "Python"),
Entwickler("Ali", 4400, "Java"),
    Manager("Tom", 5800, 6), Mitarbeiter("Sara",
3600),
```

```
    Mitarbeiter("Max", 3400), Manager("Eva", 6100,
3),
]
print(personalkosten(team))
team.append(Werkstudent("Jonas", 1300, 20))
team.append(Werkstudent("Mia", 1100, 16))
print(personalkosten(team), team[-2].arbeiten())
```

Ausgabe
```
28850
31250 Jonas arbeitet 20 Stunden pro Woche mit.
```

Runde	Lena (E)	Ali (E)	Tom (M, 6)	Sara	Max	Eva (M, 3)	+ Jonas, Mia (W)
`monatskosten()`	4200 (geerbt)	4400	5800 + 900 = 6700	3600	3400	6100 + 450 = 6550	1300, 1100 (geerbt)
`summe` danach	4200	8600	15300	18900	22300	**28850**	**31250**

`personalkosten()` muss für die Werkstudenten **nicht** geändert werden. Genau das ist der Nutzen der Polymorphie. Zusatz der Übung: Ein `Freelancer` ohne Vererbung, aber mit `monatskosten()` läuft (Duck Typing), ty meldet `invalid-argument-type`.

6.4 `object` und `__repr__`

`object` ist die Wurzel jeder Klasse, auch ohne Klammer (F. 42/43). Von dort kommen schlichte Standardversionen von `__str__`, `__repr__` und `__eq__`: `print` zeigt Klasse und Speicheradresse (die Adresse ändert sich bei jedem Lauf), `==` prüft nur, ob es *dasselbe* Objekt ist. Wer in VL5 `__str__` geschrieben hat, hat also schon eine Methode von `object` überschrieben.

```
class Leer:
    pass

print(Leer())
```

Ausgabe
```
<__main__.Leer object at 0x7f0676aceba0>
```

`__repr__` ist die Beschreibung für Entwickler, im Format wie bei `@dataclass`: `Klasse(attribut=wert, …)` (F. 43/44). `__str__` ist für Menschen (Kontoauszug).

- `type(self).__name__` liefert den Namen der **tatsächlichen** Klasse. So passt ein einziges `__repr__` in der Oberklasse auch für jede Unterklasse.
- `!r` im f-String setzt Texte in Anführungszeichen (`'Anna'`), Zahlen bleiben ohne.
- Gibt es kein `__str__`, nimmt `print()` das `__repr__`. Listen zeigen ihre Elemente immer mit `__repr__`.

```
from typing import override

class Leer:
    pass

class Konto:
    def __init__(self, inhaber: str, kontostand:
int) -> None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    @override
    def __repr__(self) -> str:
        return f"{type(self).__name__}(inhaber=
{self.inhaber!r}, kontostand={self.kontostand})"

class Sparkonto(Konto):
    pass

print(Leer.__bases__)
print(Leer() == Leer())
print(Konto("Anna", 1000))  # kein __str__ → print
nimmt __repr__
print(Sparkonto("Ben", 50))
print([Konto("Cem", 7)])  # Listen zeigen immer
__repr__
```

Ausgabe
```
(<class 'object'>,)
False
Konto(inhaber='Anna', kontostand=1000)
Sparkonto(inhaber='Ben', kontostand=50)
[Konto(inhaber='Cem', kontostand=7)]
```

6.5 Abstrakte Klassen: `ABC` und `@abstractmethod`

Was soll `Zahlungsart().bezahlen(…)` eigentlich tun? Nichts Sinnvolles: Eine „Zahlungsart an sich“ gibt es nicht, nur Kreditkarte, Rechnung usw. `Zahlungsart` ist eine **Abstraktion**. Eine **abstrakte Klasse** ist wie ein **Formular mit Pflichtfeldern**: Sie legt fest, *welche* Methoden jede Unterklasse haben **muss**, aber nicht *wie*. Objekte von ihr gibt es **nicht** (F. 45–47). Bild im Kopf: Im Café bestellst du kein „Getränk“, sondern Kaffee oder Tee.

```
from abc import ABC, abstractmethod
from decimal import Decimal
from typing import override

class Zahlungsart(ABC):  # Zutat 1: erbt von ABC
    @abstractmethod  # Zutat 2: Pflichtmethode
    def bezahlen(self, betrag: Decimal) -> str:
        """Jede konkrete Zahlungsart muss das umsetzen."""

    def beleg(self, betrag: Decimal) -> str:  # konkret, für
alle gleich
        return f"Beleg: {self.bezahlen(betrag)}"
```

- **Zwei Zutaten, beide nötig:** erben von `ABC` **und** `@abstractmethod`, beides aus dem Modul `abc`. Ohne `(ABC)` wirkt `@abstractmethod` nicht: `Geraet()` lässt sich dann ganz normal erzeugen.
- Die abstrakte Methode braucht keinen echten Rumpf: ein Docstring oder `...` genügt.

```
class Rechnungskauf(Zahlungsart):
    @override
    def bezahlen(self, betrag: Decimal) -> str:
        return f"Rechnung über {betrag} Euro, zahlbar in 14
Tagen"

class PayPal(Zahlungsart):  # bezahlen vergessen!
    def __init__(self, email: str) -> None:
        self.email = email

print(Rechnungskauf().beleg(Decimal("80.00")))
p = PayPal("anna@example.com")  # Fehler beim ERZEUGEN
```

Ausgabe

```
Beleg: Rechnung über 80.00 Euro, zahlbar in 14 Tagen
TypeError: Can't instantiate abstract class PayPal without an
implementation for abstract method 'bezahlen'
```

- Der Fehler kommt schon beim **Erzeugen** des Objekts, nicht erst beim Aufruf von `bezahlen`. `Zahlungsart()` ergibt denselben `TypeError` für `Zahlungsart`.
- `ty` meldet das schon **vor dem Start**: `call-non-callable: Cannot instantiate abstract class`.
- Eine Unterklasse, die **nicht alle** abstrakten Methoden überschreibt, bleibt selbst abstrakt (`PayPal`). Eine **konkrete** Klasse setzt alle um, von ihr gibt es Objekte.
- Abstrakt und konkret dürfen gemischt werden: `beleg()` ruft das jeweilige `bezahlen()` auf (wieder Polymorphie).
- Java: `abstract class` … `extends`. Python: `class X(ABC):` + `@abstractmethod`.

VL6 Übung 4 (Geräteverwaltung, Teil 1): abstrakte Klasse `Geraet(ABC)` mit `hersteller`, `modell`, `baujahr`, konkreter Methode `einschalten()` und abstrakter Methode `status() -> str`; `__repr__` nur einmal in `Geraet` mit `type(self).__name__`. Unterklassen `Laptop` und `Smartphone` (`akku_prozent`) sowie `Router` (`verbundene_geraete`). Ausgabe laut Musterlösung z. B. `Laptop(hersteller='Lenovo', modell='ThinkPad T14', baujahr=2023) - Akku 80 %` und `Lenovo ThinkPad T14 wird eingeschaltet.` `Geraet(…)` direkt: ty `call-non-callable`, Python `TypeError: Can't instantiate abstract class Geraet …`. Eigene Methoden wie `screenshot()` in einer `list[Geraet]` nur nach `isinstance(geraet, Smartphone)`.

6.6 Fähigkeiten statt Interfaces

Rechnung, Lieferschein und Newsletter *sind* Dokumente: eine **Kategorie** (Oberklasse). „Kann gedruckt werden“ und „kann versendet werden“ sind **Fähigkeiten**, die quer durch die Klassen gehen (F. 50). Java erlaubt nur eine Oberklasse und nimmt dafür `interface` + `implements`. Python hat kein `interface`, erlaubt aber **mehrere Oberklassen**. Eine Fähigkeit ist eine **abstrakte Klasse mit nur abstrakten Methoden**, ohne Attribute und ohne `__init__` (F. 51/52). In der Klammer steht die **Kategorie zuerst**, dann die Fähigkeiten.

```
from abc import ABC, abstractmethod
from typing import override

class Druckbar(ABC):  # Fähigkeit
    @abstractmethod
    def drucken(self) -> None: ...

class Versendbar(ABC):  # Fähigkeit
    @abstractmethod
    def versenden(self, email: str) -> None: ...

class Dokument:  # Kategorie
    def __init__(self, nummer: str) -> None:
        self.nummer = nummer

class Rechnung(Dokument, Druckbar, Versendbar):  # Kategorie
zuerst
    @override
    def drucken(self) -> None:
        print(f"Rechnung {self.nummer} wird gedruckt")

    @override
    def versenden(self, email: str) -> None:
        print(f"Rechnung {self.nummer} geht an {email}")

class Lieferschein(Dokument, Druckbar):  # kann nur drucken
    @override
    def drucken(self) -> None:
        print(f"Lieferschein {self.nummer} wird gedruckt")

class Newsletter(Dokument, Versendbar):  # kann nur versenden
    @override
    def versenden(self, email: str) -> None:
        print(f"Newsletter {self.nummer} geht an {email}")
```

Rundentabelle (F. 52–55):

Runde	`dokument`	Druckbar?	Versendbar
1	Rechnung R-1001	ja → gedruckt	ja → geht ar …
2	Lieferschein L-2001	ja → gedruckt	nein → nicht
3	Newsletter N-11	nein → nichts	ja → geht ar …

Ohne die `isinstance`-Prüfung gäbe es in Runde 2 `AttributeError: 'Lieferschein' object has no attribute 'versenden'`. Kommt eine neue Klasse `Mahnung` dazu, bleibt die Schleife unverändert.

(F. 56/57)	Abstrakte Klasse (Kategorie)	Fähigkeit (Interface)
Beziehung	„ist ein …“	„kann …“
Methoden	konkrete und abstrakte	nur abstrakte
Attribute, `__init__`	ja	nein

```
ablage: list[Dokument] = [Rechnung("R-1001"), Lieferschein("L-2001"), Newsletter("N-11")]
for dokument in ablage:
    if isinstance(dokument, Druckbar):
        dokument.drucken()
    if isinstance(dokument, Versendbar):
        dokument.versenden("einkauf@kunde.de")
```

Ausgabe

```
Rechnung R-1001 wird gedruckt
Rechnung R-1001 geht an einkauf@kunde.de
Lieferschein L-2001 wird gedruckt
Newsletter N-11 geht an einkauf@kunde.de
```

wie viele je Klasse?	eine	beliebig viele
Java	`abstract class`, `extends`	`interface`, `implements`

Gemeinsam: Beide erben von `ABC`, von beiden gibt es keine Objekte. Mehrere Oberklassen nutzt die Vorlesung **nur** für Fähigkeiten. Ausblick: `typing.Protocol` erfüllt schon, wer die Methoden hat, ohne zu erben (Duck Typing, das ty prüft); ob das Klausurstoff ist, ist offen.

VL6 Übung 5 (Geräteverwaltung, Teil 2): Fähigkeiten `Aufladbar` (`aufladen()`) und `Netzwerkfaehig` (`verbinden(netzwerk)`). `class Laptop(Geraet, Aufladbar, Netzwerkfaehig)`, `Router(Geraet, Netzwerkfaehig)`, `Monitor(Geraet)` ohne Fähigkeit und ohne eigenes `__init__`. In der Schleife lädt `isinstance(geraet, Aufladbar)` nur Laptop und Smartphone (Akku 100 %), der Monitor bekommt weder Ladung noch Fehler. `alle_aufladen(inventar) -> int` zählt mit `anzahl += 1` nur im `if`-Zweig und liefert `2`.

6.7 Wiederholung VL1–VL6: Fitnessstudio

Seit 09.11. in Moodle (15 Folien und 7 Python-Dateien). Ein Beispiel zieht sich durch alle Vorlesungen, zu jeder gibt es eine Theorie- und eine Beispielfolie mit „Was wird ausgegeben?“. Ob sie am 06.11. gezeigt wurde, ist unklar.

VL	was wiederholt wird
1	Check-in als Algorithmus: terminierend, determiniert; unlesbare Karte nicht abgedeckt → **nicht vollständig**
2	Datentypen, Geld als `Decimal("19.90")`, `//` und `%`
3	Minuten aufsummieren; `if`/`elif`: „Stammgast“ ab 8, „Regelmäßig dabei“ ab 4 Besuchen; der erste wahre Zweig gewinnt
4	`monatsbeitrag(tarif, studierend=False)`, Schlüsselwortargument; `gewicht(woche)` rekursiv mit Basisfall
5	`Mitgliedschaft` mit Property `beitrag`, Setter lehnt negative Werte ab
6	`Mitgliedschaft(ABC)`, `Basic`, `Premium`, `Student(Basic)` mit `super()` − 5 Euro

Eigenes Beispiel im gleichen Stil:

```
from abc import ABC, abstractmethod
from decimal import Decimal

class Tarif(ABC):
    def __init__(self, name: str) -> None:
        self.name = name
        self.besuche: list[int] = []  # Minuten je Besuch

    @abstractmethod
    def monatsbeitrag(self) -> Decimal: ...

    def einchecken(self, minuten: int) -> None:
        self.besuche.append(minuten)

class Basic(Tarif):
    def monatsbeitrag(self) -> Decimal:
        return Decimal("24.90")

class Student(Basic):
    def monatsbeitrag(self) -> Decimal:
        return super().monatsbeitrag() - Decimal("5.00")

def gewicht(woche: int) -> float:
    if woche == 1:  # Basisfall
        return 40.0
    return gewicht(woche - 1) + 2.5

mia = Student("Mia")
for minuten in [50, 70, 40]:
    mia.einchecken(minuten)
print(mia.name, mia.monatsbeitrag(),
len(mia.besuche), sum(mia.besuche))
print(gewicht(3))
print(0.1 + 0.2, Decimal("0.1") + Decimal("0.2"))
print(130 // 60, 130 % 60)
```

Ausgabe

```
Mia 19.90 3 160
45.0
0.30000000000000004 0.3
2 10
```

`Student` holt mit `super().monatsbeitrag()` den Basic-Preis: 24.90 − 5.00 = 19.90. `gewicht(3)` = 40 + 2,5 + 2,5. `gewicht(0)` erreicht den Basisfall nie → `RecursionError`; sicherer ist `if woche <= 1:`. `Tarif("Max")` → `TypeError` (abstrakt).

Typische Fehler und Fallen

- `super().__init__(…)` im eigenen `__init__` vergessen → `AttributeError` erst später; ty meldet nichts. `super.__init__` ohne Klammern → `TypeError`.
- In der Unterklasse ein geerbtes Attribut versehentlich neu belegen (`self.kontostand = laufzeit`): Ober- und Unterklasse teilen sich **dasselbe** Attribut (A.2).
- Überladen und Überschreiben verwechseln (Klausurklassiker). `from typing import override` vergessen → `NameError`.
- Vererbung für „hat ein“ benutzen, mehrere Oberklassen für mehrere Kategorien statt nur für Fähigkeiten.
- Unterklassen-Attribut über die Oberklassen-Typangabe benutzen (`zahlungsart.nummer`) → ty `unresolved-attribute`; erst `isinstance`.
- „Duck Typing heißt, ty ist egal“: Das Programm läuft, ty meldet trotzdem `invalid-argument-type`.
- Nur eine Zutat für abstrakt (`@abstractmethod` ohne `(ABC)`) → Python verhindert das Erzeugen nicht.
- Den `TypeError` einer abstrakten Klasse „erst beim Aufruf“ erwarten: Er kommt beim **Erzeugen**.
- `isinstance` vor einer Fähigkeits-Methode vergessen → `AttributeError`. `__repr__` mit festem Klassennamen statt `type(self).__name__`.

Das musst du können

☐ „ist ein“ von „hat ein“ unterscheiden und eine Unterklasse mit `super().__init__(…)` schreiben
☐ eine Methode überschreiben, mit `super().methode()` ergänzen und `@override` erklären; Überladen und Überschreiben abgrenzen
☐ für einen Aufruf vorhersagen, welche Methode läuft (Methodensuche, `__mro__`), und `isinstance` / `type` auswerten
☐ Polymorphie an einer Schleife oder Funktion über eine Liste der Oberklasse erklären; Typangabe vs. Objekt, Duck Typing
☐ `__repr__` mit `type(self).__name__` und `!r` schreiben; sagen, was `object` liefert
☐ eine abstrakte Klasse mit `ABC` und `@abstractmethod` schreiben und sagen, wann welcher Fehler kommt
☐ Fähigkeiten als abstrakte Klassen mit mehreren Oberklassen umsetzen und mit `isinstance` prüfen

VL7 Projekt Kaffeeshop

Stand und Herkunft

VL7 (Folien „vl07_projekt_onlineshop“, 23 Folien) bringt keinen neuen Stoff, sondern ein Projekt. Es beginnt laut Plan am **06.11.** mit Stufe 1; Stufe 2 und 3 folgen eher am nächsten Termin. Eine offizielle Musterlösung gibt es erst nach dem Projekt (F. 22). Der Code hier ist deshalb eine **eigene Lösung** der Folienaufgaben, `Produkt` ist eine vereinfachte Fassung der Datei aus dem Startpaket. **Für die Klausur** zählen die Muster darin: ein Element in einer Liste finden, Treffer sammeln, aufsummieren, Bestand prüfen, ein Objekt über zwei Namen verändern.

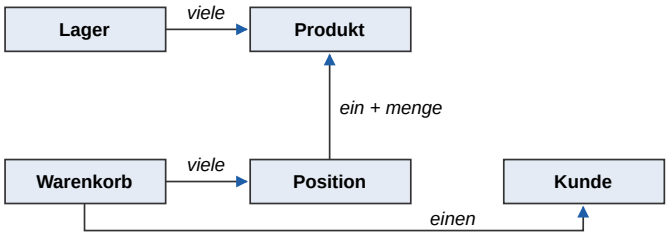
7.1 Aufgabe, Ablauf und Startpaket

Zu zweit entsteht ein kleiner Onlineshop für Kaffee und Zubehör in der Kommandozeile: Sortiment ansehen, suchen, in den Warenkorb legen, bestellen. Dabei kommt alles zusammen: Klassen, Properties, `@dataclass` (VL5), Listen, Schleifen, `match` (VL3), Funktionen und Module (VL4/5), Vererbung, abstrakte Klassen, Polymorphie (VL6) (F. 3). **Ziel: Am Ende läuft bei jedem Team mindestens Stufe 1.**

Ablauf (F. 4): Aufgabe und Startpaket → **Stufe 1** Lager, Warenkorb, Bestellung (Pflicht) → Zwischenstand (zwei Teams zeigen) → **Stufe 2** Produktarten und Zahlungsarten → **Stufe 3** Zusatz nach Wahl → Vorstellung (je ca. 5 Min.) und Musterlösung.

Pair Programming (F. 5): ein Rechner, eine Person tippt, die andere liest und denkt mit; etwa alle 30 Minuten wechseln; **beide** müssen jede Zeile erklären können. Weiter ohne KI, prüfen mit `uv run ruff check .` und `uv run ty check`.

Datei (F. 8)	Inhalt	du
`produkt.py`	`Produkt` aus VL5 Übung 4 plus Artikelnummer: Properties `preis`, `bestand`, `bruttopreis` (19 %), `einlagern()`, `verkaufen()`	benutzen
`daten.py`	`beispielsortiment()`: 8 Produkte, 101–104 Lebensmittel, 201–204 Zubehör	benutzen
`main.py`	Menü mit `while True` + `match`, Punkt 1 und 0, fertige Funktion `zahl_einlesen()`	ausbauen
`lager.py`, `warenkorb.py`	gibt es noch nicht	schreiben



Stufe 1: lauter Hat-ein-Beziehungen, keine Vererbung (F. 7).

Die vereinfachte `produkt.py`, mit der alle Beispiele dieses Kapitels laufen. `daten.py` legt damit acht Produkte an, z. B. `Produkt(101, "Kaffee Hausmischung 1 kg", Decimal("12.99"), 40)` und `Produkt(203, "French Press 1 l", Decimal("20.90"), 3)`.

Bild im Kopf: Das Lager ist ein Regal voller Produkte. Im Warenkorb liegen keine Produkte direkt, sondern Zettel (Positionen): „3 × French Press“. Ein Zettel hat ein Produkt und eine Menge.

Bruttopreis: 12.99 · 1.19 = 15.4581 → `quantize(Decimal("0.01"))` → 15.46. French Press: 20.90 · 1.19 = 24.871 → 24.87.

```
from decimal import Decimal
from typing import Final

MWST_SATZ: Final = Decimal("0.19")

class Produkt:
    def __init__(self, nummer: int, name: str,
preis: Decimal, bestand: int) -> None:
        self.nummer = nummer
        self.name = name
        self._preis = preis
        self._bestand = bestand

    @property
    def bestand(self) -> int:
        return self._bestand

    @property
    def bruttopreis(self) -> Decimal:
        return (self._preis * (1 +
MWST_SATZ)).quantize(Decimal("0.01"))

    def einlagern(self, menge: int) -> None:
        self._bestand += menge

    def verkaufen(self, menge: int) -> bool:
        if menge > self._bestand:
            return False
        self._bestand -= menge
        return True
```

```
    def __str__(self) -> str:
        return f"{self.nummer}  {self.name}:
{self.bruttopreis} Euro ({self._bestand} auf
Lager)"
```

7.2 Stufe 1, Teil 1: Das Lager (finden und suchen)

Aufgabe (F. 10): `Lager(produkte)` mit `anzeigen()`; `finden(nummer) -> Produkt | None` liefert das Produkt mit dieser Artikelnummer oder `None`; `suchen(text) -> list[Produkt]` liefert alle Produkte, deren Name den Text enthält, ohne Rücksicht auf Groß- und Kleinschreibung. In `main()` ein `Lager(beispielsortiment())` anlegen und Menüpunkt 1 und 2 damit bauen.

```
from produkt import Produkt

class Lager:
    def __init__(self, produkte: list[Produkt]) ->
None:
        self.produkte = produkte

    def anzeigen(self) -> None:
        for produkt in self.produkte:
            print(produkt)

    def finden(self, nummer: int) -> Produkt |
None:
        for produkt in self.produkte:
            if produkt.nummer == nummer:
                return produkt  # gefunden: sofort
raus
        return None  # erst NACH der Schleife

    def suchen(self, text: str) -> list[Produkt]:
        treffer: list[Produkt] = []
        for produkt in self.produkte:
            if text.lower() in
produkt.name.lower():
                treffer.append(produkt)
        return treffer
```

```
from daten import beispielsortiment
from lager import Lager

lager = Lager(beispielsortiment())
print(lager.finden(203))
print(lager.finden(999))
for produkt in lager.suchen("kaffee"):
    print(produkt.name)
print(lager.suchen("Zucker"))
```

Ausgabe

```
203  French Press 1 l: 24.87 Euro (3 auf Lager)
None
Kaffee Hausmischung 1 kg
Kaffeemühle mit Handkurbel
[]
```

Rundentabelle für `finden(203)` (Reihenfolge 101, 102, 103, 104, 201, 202, 203, 204):

Runde	`produkt.nummer`	`== 203 ?`	was passiert
1–6	101 … 202	nein	weiter
7	203	ja	`return produkt`, Methode endet sofort, Runde 8 gibt es nicht

Bei `finden(999)` ist die Antwort in allen acht Runden „nein“, erst **nach** der Schleife kommt `return None`. `Produkt | None` heißt: entweder ein Produkt oder nichts. Wer `finden()` aufruft, muss `None` abfangen (`if produkt is None:`), sonst `AttributeError: 'NoneType' object has no attribute 'name'`.

Muster	vor der Schleife	in der Schleife	nach der Schleife
finden (erstes)	–	Treffer → sofort `return`	`return None`
sammeln (alle)	`treffer = []`	Treffer → `append`	`return treffer`
summieren	`summe = 0`	`summe += …`	`return summe`
zählen	`anzahl = 0`	Treffer → `+= 1`	`return anzahl`

`suchen("kaffee")`: `.lower()` macht beide Texte klein, `in` prüft, ob der Text im Namen steckt. Treffer: 101 und 201. Kein Treffer ergibt `[]`, nicht `None`; die Schleife im Menü läuft dann null Runden.

Häufigster Fehler: `else: return None` direkt unter dem `if`. Dann endet die Suche nach dem ersten falschen Produkt, 203 wird nie gefunden (A.2). `return None` gehört eine Stufe weniger eingerückt hinter die Schleife.

7.3 Stufe 1, Teil 2: Der Warenkorb und `hinzufuegen()`

Aufgabe (F. 12): `Kunde` und `Position` aus VL5 Übung 5 übernehmen, `summe()` rechnet jetzt mit dem `bruttopreis`. `Warenkorb(kunde)` mit leerer Liste `positionen`. `hinzufuegen(produkt, menge) -> bool` liefert `False`, wenn die Menge nicht auf Lager ist; **was schon im Korb liegt, zählt mit**. Liegt das Produkt schon im Korb, wird die **Menge der vorhandenen Position erhöht** (keine zweite Position). Dazu `entfernen(nummer) -> bool`, `gesamtsumme()`, `ist_leer()`, `anzeigen()`. `position_zu()` und `menge_im_korb()` sind eigene Hilfsmethoden, gebaut wie `finden()`.

```
from dataclasses import dataclass
from decimal import Decimal

from produkt import Produkt

@dataclass
class Kunde:
    name: str
    email: str

@dataclass
class Position:
    produkt: Produkt
    menge: int

    def summe(self) -> Decimal:
        return self.produkt.bruttopreis * self.menge

class Warenkorb:
    def __init__(self, kunde: Kunde) -> None:
        self.kunde = kunde
        self.positionen: list[Position] = []

    def position_zu(self, nummer: int) -> Position |
None:
        for position in self.positionen:
            if position.produkt.nummer == nummer:
                return position
        return None

    def menge_im_korb(self, nummer: int) -> int:
        position = self.position_zu(nummer)
        if position is None:
            return 0
        return position.menge
```

... Fortsetzung der Klasse `Warenkorb`:

```
    def hinzufuegen(self, produkt: Produkt, menge: int) -
> bool:
        schon_im_korb =
self.menge_im_korb(produkt.nummer)
        if schon_im_korb + menge > produkt.bestand:
            return False  # zu wenig da
        position = self.position_zu(produkt.nummer)
        if position is None:  # neuer Zettel
            neu = Position(produkt, menge)
            self.positionen.append(neu)
        else:  # vorhandenen erhöhen
            position.menge += menge
        return True

    def entfernen(self, nummer: int) -> bool:
        position = self.position_zu(nummer)
        if position is None:
            return False
        self.positionen.remove(position)
        return True

    def gesamtsumme(self) -> Decimal:
        summe = Decimal(0)
        for position in self.positionen:
            summe += position.summe()
        return summe

    def ist_leer(self) -> bool:
        return len(self.positionen) == 0

    def anzeigen(self) -> None:
        print(f"Warenkorb von {self.kunde.name}")
        for position in self.positionen:
            print(f"{position.menge} x
{position.produkt.name}: {position.summe()} Euro")
        print(f"Summe: {self.gesamtsumme()} Euro")
```

```
from daten import beispielsortiment
from lager import Lager
from warenkorb import Kunde, Warenkorb

lager = Lager(beispielsortiment())
korb = Warenkorb(Kunde("Anna Berger", "anna.berger@example.com"))
fp = lager.finden(203)  # French Press, Bestand 3
kaffee = lager.finden(101)  # Bestand 40
assert fp is not None and kaffee is not None
print(korb.hinzufuegen(fp, 5), korb.hinzufuegen(fp, 3), korb.hinzufuegen(fp, 1))
print(korb.hinzufuegen(kaffee, 2), korb.hinzufuegen(kaffee, 1))
print(len(korb.positionen))
korb.anzeigen()
```

Ausgabe

```
False True False
True True
2
Warenkorb von Anna Berger
3 x French Press 1 l: 74.61 Euro
3 x Kaffee Hausmischung 1 kg: 46.38 Euro
Summe: 120.99 Euro
```

`hinzufuegen()` in drei Schritten: (1) Was liegt schon im Korb? (2) Reicht der Bestand für **beides zusammen**? Wenn nicht: `return False`, nichts wird geändert. (3) Zettel neu anhängen oder vorhandene Menge erhöhen, dann `return True`.

Am Ende **zwei** Positionen, nicht drei. `position.menge += menge` ändert das Objekt, das in der Liste liegt: Name und Listeneintrag zeigen auf **dasselbe** Objekt (Kap. 5.4). `gesamtsumme()`: 0 → 74.61 (3 · 24.87) → 120.99 (+ 3 · 15.46).

`hinzufuegen(…)`	schon im Korb	Prüfung	Ergebnis
fp, 5	0	0 + 5 > 3 → ja	`False`
fp, 3	0	0 + 3 > 3 → nein	`True`, neue Position
fp, 1	3	3 + 1 > 3 → ja	`False`
kaffee, 2	0	0 + 2 > 40 → nein	`True`, neue Position
kaffee, 1	2	2 + 1 > 40 → nein	`True`, Menge 2 → 3

Die zwei typischen Fehler in `hinzufuegen()`

Nur `menge > produkt.bestand` prüfen und die Menge im Korb vergessen: Dann gehen „erst 3, dann noch 1 French Press“ beide durch, im Korb liegen 4 von 3. Oder immer `append`, ohne nach einem vorhandenen Zettel zu schauen: Dann steht Kaffee zweimal als zwei Positionen im Korb. `hinzufuegen()` liefert nur `True` / `False`, die Meldung schreibt `main.py` (siehe 4.9).

7.4 Stufe 1, Teil 3: Das Menü in `main.py`

Aufgabe (F. 11/13): am Anfang Name und E-Mail abfragen → `Kunde` + `Warenkorb`. Menü 3 „In den Warenkorb“ (Nummer und Menge mit `zahl_einlesen()`, Meldung bei unbekannter Nummer oder zu großer Menge), 4 anzeigen, 5 entfernen, 6 bestellen (für jede Position `verkaufen()`, Summe ausgeben, Korb leeren; leerer Korb → nicht bestellbar). Tipp der Folie: **je Menüpunkt eine eigene Funktion**, dann bleibt die Schleife in `main()` kurz. Gekürzt (ohne Menütext, Punkte 2 und 5 weggelassen):

```
from daten import beispielsortiment
from lager import Lager
from warenkorb import Kunde, Warenkorb

def zahl_einlesen(frage: str) -> int:  # fertig (Startpaket)
    while True:
        eingabe = input(frage)
        if eingabe.isdigit():  # nur Ziffern?
            return int(eingabe)
        print("Bitte eine ganze Zahl eingeben.")

def in_den_warenkorb(lager: Lager, korb: Warenkorb) -> None:
    produkt = lager.finden(zahl_einlesen("Artikelnummer: "))
    if produkt is None:
        print("Diese Artikelnummer gibt es nicht.")
        return  # nur dieser Menüpunkt endet
    menge = zahl_einlesen("Menge: ")
    if korb.hinzufuegen(produkt, menge):
        print(f"{menge} x {produkt.name} liegt im Warenkorb.")
    else:
        print(f"Das geht nicht. Auf Lager: {produkt.bestand}, "
              f"schon im Warenkorb:
{korb.menge_im_korb(produkt.nummer)}.")

def bestellen(korb: Warenkorb) -> None:
    if korb.ist_leer():
        print("Der Warenkorb ist leer.")
        return
    for position in korb.positionen:
        position.produkt.verkaufen(position.menge)
    print(f"Vielen Dank, {korb.kunde.name}! "
          f"Ihre Bestellung über {korb.gesamtsumme()} Euro.")
    korb.positionen = []  # neue Liste (Stufe 2!)

def main() -> None:
    # alles VOR der Schleife anlegen
    lager = Lager(beispielsortiment())
    kunde = Kunde(input("Ihr Name: "), input("Ihre E-Mail: "))
    korb = Warenkorb(kunde)
    while True:
        match input("Ihre Wahl: "):
            case "1":
                lager.anzeigen()
            case "3":
                in_den_warenkorb(lager, korb)
            case "4":
                korb.anzeigen()
            case "6":
                bestellen(korb)
            case "0":
                print("Auf Wiedersehen!")
                break
            case _:
                print("Bitte eine Zahl aus dem Menü eingeben.")

if __name__ == "__main__":
    main()
```

Ablauf am Terminal (echter Lauf, die Eingaben sind mit abgedruckt):

Ausgabe

```
Ihr Name: Anna Berger
Ihre E-Mail: anna@example.com
Ihre Wahl: 3
Artikelnummer: abc
Bitte eine ganze Zahl eingeben.
Artikelnummer: 999
Diese Artikelnummer gibt es nicht.
Ihre Wahl: 3
Artikelnummer: 203
Menge: 3
3 x French Press 1 l liegt im Warenkorb.
Ihre Wahl: 3
Artikelnummer: 203
Menge: 1
Das geht nicht. Auf Lager: 3, schon im
Warenkorb: 3.
Ihre Wahl: 3
Artikelnummer: 101
Menge: 2
2 x Kaffee Hausmischung 1 kg liegt im
Warenkorb.
Ihre Wahl: 4
Warenkorb von Anna Berger
3 x French Press 1 l: 74.61 Euro
2 x Kaffee Hausmischung 1 kg: 30.92 Euro
Summe: 105.53 Euro
Ihre Wahl: 6
Vielen Dank, Anna Berger! Ihre Bestellung über
105.53 Euro.
Ihre Wahl: 6
Der Warenkorb ist leer.
Ihre Wahl: 7
Bitte eine Zahl aus dem Menü eingeben.
Ihre Wahl: 0
Auf Wiedersehen!
```

Runde	`eingabe`	`.isdigit()`	was passiert
1	`"abc"`	False	Meldung, nächste Runde
2	`"999"`	True	`return 999`: Schleife und Funktion enden

- `zahl_einlesen()`: `while True` läuft „für immer“, `return` ist der einzige Ausgang. Deshalb kein `ValueError` bei `abc`. Auch `-3` lehnt `isdigit()` ab.
- Lager, Kunde und Korb werden **einmal vor** der `while`-Schleife angelegt. Stünde `korb = Warenkorb(kunde)` in der Schleife, wäre der Korb nach jeder Menüwahl leer, wie bei `summe = 0` in der Schleife.
- Ein frühes `return` in einer Menüfunktion heißt nur „Schluss für diesen Menüpunkt“, das Menü läuft weiter. `break` im `case "0"` beendet die Schleife; `case _` fängt alles andere ab.

- `Kunde(input(…), input(…))` fragt erst den Namen, dann die E-Mail ab: Argumente werden von links nach rechts ausgewertet.
- **Testfälle von F. 13/14:** Suche „kaffee“ → 2 Treffer · 5 × French Press → abgelehnt · erst 3, dann 1 → das zweite Mal abgelehnt · zweimal Kaffee → eine Position · 999 oder `abc` → Meldung, kein Absturz · nach dem Bestellen Korb leer, Bestand kleiner · ruff und ty melden nichts.

7.5 Stufe 2: Produktarten und Zahlungsarten

Jetzt kommt VL6 dazu. Für die meisten Lebensmittel gilt in Deutschland der ermäßigte Mehrwertsteuersatz von 7 %, auch für Kaffee und Tee (F. 16). `Produkt` wird abstrakt mit der abstrakten Methode `mwst_satz()`, `bruttopreis` rechnet mit `self.mwst_satz()`. Unterklassen: `Lebensmittel` (7 %) und `Zubehoer` (19 %). Kontrollwerte der Folie: Kaffee Hausmischung **13.90** statt 15.46 (12.99 · 1.07 = 13.8993), French Press bleibt **24.87**.

Das ist Polymorphie: `bruttopreis` steht nur einmal in `Produkt`, welches `mwst_satz()` läuft, entscheidet das Objekt. Lager, Warenkorb und Menü ändern sich dafür nicht, `list[Produkt]` passt weiter, weil ein Lebensmittel ein Produkt ist. Ein „Produkt an sich“ lässt sich nicht mehr anlegen (`TypeError`).

Zahlungsarten (F. 17): `zahlungsarten.py` mit abstrakter `Zahlungsart` und `Kreditkarte`, `Lastschrift`, `Rechnungskauf` wie in 6.3/6.5; `@dataclass Bestellung` mit `nummer`, `kunde`, `positionen`, `zahlungsart`; die Bestellungen kommen in eine Liste in `main()`, Menüpunkt 7 „Meine Bestellungen“ zeigt sie (F. 19).

```
from abc import ABC, abstractmethod
from decimal import Decimal
from typing import override

class Produkt(ABC):
    def __init__(self, nummer: int, name: str,
preis: Decimal) -> None:
        self.nummer = nummer
        self.name = name
        self._preis = preis

    @abstractmethod
    def mwst_satz(self) -> Decimal: ...

    @property
    def bruttopreis(self) -> Decimal:  # steht nur
EINMAL hier
        return (self._preis * (1 +
self.mwst_satz())).quantize(Decimal("0.01"))

class Lebensmittel(Produkt):
    @override
    def mwst_satz(self) -> Decimal:
        return Decimal("0.07")

class Zubehoer(Produkt):
    @override
    def mwst_satz(self) -> Decimal:
        return Decimal("0.19")

sortiment: list[Produkt] = [
    Lebensmittel(101, "Kaffee Hausmischung 1 kg",
Decimal("12.99")),
    Zubehoer(203, "French Press 1 l",
Decimal("20.90")),
]
for produkt in sortiment:
    print(produkt.nummer, produkt.name,
produkt.bruttopreis)
```

Ausgabe

```
101 Kaffee Hausmischung 1 kg 13.90
203 French Press 1 l 24.87
```

Achtung beim Leeren des Warenkorbs (F. 18)

`Bestellung(1, korb.kunde, korb.positionen, …)` kopiert die Liste **nicht**. Korb und Bestellung zeigen danach auf **dieselbe** Liste.
`korb.positionen.clear()` leert deshalb auch die Bestellung. Richtig ist `korb.positionen = []`: Der Korb bekommt eine neue Liste, die alte gehört nur noch der Bestellung. Vereinfacht mit Strings:

```
korb_positionen = ["2 x Kaffee"]
bestellung_positionen = korb_positionen  # keine
Kopie!
korb_positionen.clear()  # leert die EINE Liste
print(bestellung_positionen)

korb_positionen = ["1 x Tee"]
bestellung2_positionen = korb_positionen
korb_positionen = []  # Korb bekommt neue Liste
print(bestellung2_positionen, korb_positionen)
```

Ausgabe

```
[]
['1 x Tee'] []
```

7.6 Stufe 3: Zusatzaufgaben nach Wahl

Wer fertig ist, sucht sich etwas aus (F. 21): **Versandkosten** (4,90 €, ab 50 € Warenwert frei), **Rabattcode mit `match`** (z. B. DHBW10: 10 %, WILLKOMMEN5: 5 € ab 25 €), **Mitarbeiterbereich** (Produkte mit weniger als 5 Stück anzeigen und mit `einlagern()` nachbestellen), **selbst sortieren** nach Preis ohne `sorted()`, **Tests mit pytest**. Die Skizzen sind eigene Lösungen.

```
from decimal import Decimal
from typing import Final

VERSAND: Final = Decimal("4.90")
VERSANDFREI_AB: Final = Decimal("50.00")

def versandkosten(warenwert: Decimal) -> Decimal:
    if warenwert >= VERSANDFREI_AB:
        return Decimal("0.00")
    return VERSAND

def rabatt(code: str, warenwert: Decimal) -> Decimal:
    match code:
        case "DHBW10":
            return (warenwert *
Decimal("0.10")).quantize(Decimal("0.01"))
        case "WILLKOMMEN5" if warenwert >= Decimal("25.00"):  #
Guard
            return Decimal("5.00")
        case _:
            return Decimal("0.00")

print(versandkosten(Decimal("30.92")),
versandkosten(Decimal("66.57")))
print(rabatt("DHBW10", Decimal("66.57")), rabatt("WILLKOMMEN5",
Decimal("30.92")))
print(rabatt("WILLKOMMEN5", Decimal("16.90")), rabatt("GRATIS",
Decimal("66.57")))
```

Ausgabe

```
4.90 0.00
6.66 5.00
0.00 0.00
```

```
def sortieren(preise: list[float]) ->
None:
    for i in range(len(preise)):  #
Platz i
        guenstigstes = i
        for j in range(i + 1,
len(preise)): # Rest durchsuchen
            if preise[j] <
preise[guenstigstes]:
                guenstigstes = j
        preise[i],
preise[guenstigstes] =
preise[guenstigstes], preise[i]
        print(f"nach Runde {i + 1}:
{preise}")

sortieren([13.90, 8.45, 3.73, 2.99])
```

Ausgabe

```
nach Runde 1: [2.99, 8.45, 3.73, 13.9]
nach Runde 2: [2.99, 3.73, 8.45, 13.9]
nach Runde 3: [2.99, 3.73, 8.45, 13.9]
nach Runde 4: [2.99, 3.73, 8.45, 13.9]
```

`case "WILLKOMMEN5" if …` ist ein `case` mit Bedingung (Guard): Er passt nur, wenn auch die Bedingung wahr ist, sonst geht es zum nächsten `case`. 10 % von 66.57 = 6.657 → 6.66. Sortieren: Die innere Schleife sucht das günstigste ab Platz `i`, die äußere zählt die Plätze; Runde 1 tauscht Platz 0 ↔ 3, Runde 2 Platz 1 ↔ 2. In der Klausur eher als „Was steht nach Runde 2 in der Liste?“.

Typische Fehler und Fallen

- `else: return None` in der Schleife von `finden()`: Suche endet nach Runde 1.
- Das Ergebnis von `finden()` nicht auf `None` prüfen → `AttributeError: 'NoneType' object has no attribute …`.
- In `hinzufuegen()` die Menge im Korb vergessen oder immer `append` statt Menge erhöhen.
- Kein Treffer bei `suchen()` ist `[]`, nicht `None`.
- Korb oder Startwert **in** der Menü- bzw. Summenschleife anlegen: wird jede Runde neu.
- `korb.positionen.clear()` leert auch die Liste der Bestellung (zwei Namen, eine Liste).
- In Methoden der Klassen `print` / `input` statt `return`: dann nicht testbar.

Das musst du können

- ☐ die Klassen aus einem Text mit „hat ein“ ableiten (Lager, Warenkorb, Position, Kunde)
- ☐ `finden()` (Rückgabe `Produkt | None`) und `suchen()` schreiben und mit einer Rundentabelle durchspielen
- ☐ `hinzufuegen()` mit Mengenprüfung (inkl. Korb-Menge) und Erhöhen einer vorhandenen Position schreiben
- ☐ ein Menü mit `while True`, `match` und einer Funktion je Menüpunkt aufbauen
- ☐ erklären, warum `= []` und `clear()` sich unterscheiden; Stufe 2 als Anwendung von Abstraktion und Polymorphie erklären

Anhang Fehlermeldungen und Selbsttest

A.1 Fehlermeldungen auf einen Blick

Aus Platzgründen ist bei langen Meldungen nur der Kern zitiert.

Fehler	Typischer Auslöser	Kapitel
`SyntaxError`	fehlende Klammer oder fehlender `:`, `x++`, `else` ohne `if`, Parameter ohne Standardwert hinter einem mit Standardwert	2, 3, 4
`IndentationError`	Block nach `:` nicht eingerückt	3
`NameError`	Name nie zugewiesen, Tippfehler bzw. Großschreibung, nur Typangabe, übersprungener Block, lokale Variable außerhalb der Funktion	2, 3, 4
`TypeError`	`"Alter: " + 36`, Pflichtargument fehlt, Tupel ändern, `self` vergessen	2, 3, 4, 5
`ValueError`	`int("zwei")`, `zip(…, strict=True)` mit unterschiedlich langen Listen	2, 3
`ZeroDivisionError`	Division durch 0 (Absicherung mit Kurzschluss)	2, 3
`IndexError`	Index außerhalb der Liste bzw. des Strings	3
`UnboundLocalError`	Zuweisung an einen globalen Namen in der Funktion (`zaehler += 1`)	4
`RecursionError`	Abbruchbedingung fehlt oder wird nie erreicht, Tiefe über etwa 1000	4
`AssertionError`	`assert` mit falscher Bedingung	4
`AttributeError`	Zuweisung an eine Property ohne Setter, nicht vorhandenes Attribut lesen	5
`AttributeError` (VL6/7)	`super().__init__` vergessen (`'Sparkonto' object has no attribute 'kontostand'`), Fähigkeits-Methode ohne `isinstance` (`'Lieferschein' object has no attribute 'versenden'`), Ergebnis von `finden()` nicht geprüft (`'NoneType' object has no attribute 'name'`)	6, 7
`TypeError` (VL6/7)	abstrakte Klasse erzeugen oder abstrakte Methode nicht umgesetzt (`Can't instantiate abstract class PayPal without an implementation for abstract method 'bezahlen'`), `super.__init__` ohne Klammern, `print` statt `return` in `__str__` (`__str__ returned non-string`), mit `None` rechnen (`unsupported operand type(s) for +: 'NoneType' and 'int'`)	4.9, 6, 7
`NameError` (VL6)	`from typing import override` bzw. `from abc import ABC, abstractmethod` vergessen	6
nur ty	`invalid-explicit-override` (Tippfehler unter `@override`), `unresolved-attribute` (`zahlungsart.nummer` ohne `isinstance`), `invalid-argument-type` (Duck Typing ohne Erben), `call-non-callable` (abstrakte Klasse erzeugen), `invalid-return-type` (Zweig ohne `return`)	4, 6
kein Fehler!	Logikfehler (`5 + 2 / 2`), falscher Startwert, Tippfehler bei einer Attributzuweisung, `int(19.99 * 100)`; `summe = 0` oder `return` in der Schleife, `else: return None` in `finden()`, `@abstractmethod` ohne `(ABC)`, `korb.positionen.clear()` statt `= []`, Unterklasse überschreibt geerbtes Attribut	2, 3, 5, 6, 7

A.2 Selbsttest: Was gibt das aus?

Überleg dir zuerst selbst eine Antwort und schau dann auf die Ausgabe darunter.

```
print(17 // 5, 17 % 5, -17 // 5, 2 ** 3 ** 2)
print(10 / 4, 10 // 4.0, int(-2.9))
print("Ab" * 2 + "c", "Hallo"[-2:], len("a b"))
```

Ausgabe

```
3 2 -4 512
2.5 2.0 -2
AbAbc lo 3
```

```
x = 0
for i in range(10, 0, -3):
    x += i
print(i, x)
print(not 0 or 5 and "", bool(" "))
```

Ausgabe

```
1 22
True True
```

```
def f(n: int) -> int:
    if n <= 1:
        return n
    return f(n - 1) + n * 2

def g(werte: list[int]) -> None:
    werte.append(len(werte))
```

```
class Zaehler:
    stand = 0

    def __init__(self) -> None:
        Zaehler.stand += 1
        self.nr = Zaehler.stand

a = Zaehler()
```

```
zahlen = [5]
g(zahlen)
g(zahlen)
print(f(3), zahlen)
```

Ausgabe
```
11 [5, 1, 2]
```

```
b = Zaehler()
c = b
c.nr = 9
print(a.nr, b.nr, Zaehler.stand, b is c)
```

Ausgabe
```
1 9 2 True
```

Hinweise: `**` wird von rechts ausgewertet (`3 ** 2` zuerst) *(Ergänzung)*. `10 // 4.0` ergibt bei einem float-Operanden auch einen float *(Ergänzung)*. `and` und `or` liefern einen der Operanden zurück, nicht unbedingt `True` oder `False` *(Ergänzung)*.

Neu: Vererbung, Polymorphie, Kaffeeshop-Muster (VL6/VL7)

```
class Mitarbeiter:
    def __init__(self, name: str, gehalt: int) ->
None:
        self.name = name
        self.gehalt = gehalt

    def arbeiten(self) -> str:
        return f"{self.name} arbeitet."

class Entwickler(Mitarbeiter):
    def arbeiten(self) -> str:
        return f"{self.name} schreibt Code."

class Manager(Mitarbeiter):
    def __init__(self, name: str, gehalt: int,
team: int) -> None:
        super().__init__(name, gehalt)
        self.team = team

team = [Mitarbeiter("Sara", 3600),
Entwickler("Lena", 4200),
        Manager("Tom", 5800, 2)]
for person in team:
    print(person.arbeiten())
```

Ausgabe
```
Sara arbeitet.
Lena schreibt Code.
Tom arbeitet.
```

```
class Konto:
    def __init__(self, inhaber: str, kontostand:
int) -> None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    def __str__(self) -> str:
        return f"Konto von {self.inhaber}:
{self.kontostand} Euro"

class Festgeld(Konto):
    def __init__(self, inhaber: str, kontostand:
int, laufzeit: int) -> None:
        super().__init__(inhaber, kontostand)
        self.kontostand = laufzeit

f = Festgeld("Anna", 5000, 12)
print(f, isinstance(f, Konto))
```

Ausgabe
```
Konto von Anna: 12 Euro True
```

```
from abc import ABC, abstractmethod

class Tier(ABC):
    @abstractmethod
    def laut(self) -> str: ...

    def vorstellen(self) -> str:
        return "Ich sage " + self.laut()

class Hund(Tier):
    def laut(self) -> str:
        return "Wuff"

class Katze(Tier):
    pass

print(Hund().vorstellen())
print(Katze().vorstellen())
```

Ausgabe
```
Ich sage Wuff
TypeError: Can't instantiate abstract class Katze
without an implementation for abstract method
'laut'
```

```
from abc import ABC, abstractmethod

class Geraet:
    def __init__(self, name: str) -> None:
        self.name = name

class Aufladbar(ABC):
    @abstractmethod
    def aufladen(self) -> None: ...

class Netzwerkfaehig(ABC):
    @abstractmethod
    def verbinden(self, netz: str) -> None: ...

class Laptop(Geraet, Aufladbar, Netzwerkfaehig):
    def aufladen(self) -> None:
        print(self.name, "geladen")
    def verbinden(self, netz: str) -> None:
        print(self.name, "->", netz)

class Router(Geraet, Netzwerkfaehig):
    def verbinden(self, netz: str) -> None:
        print(self.name, "->", netz)

class Monitor(Geraet):
    pass

for g in [Laptop("Laptop"), Router("Router"),
Monitor("Monitor")]:
```

```
    if isinstance(g, Aufladbar):
        g.aufladen()
    if isinstance(g, Netzwerkfaehig):
        g.verbinden("DHBW-Gast")
```

Ausgabe

```
Laptop geladen
Laptop -> DHBW-Gast
Router -> DHBW-Gast
```

```
def finden(nummern: list[int], gesucht: int) -> int
| None:
    for i in range(len(nummern)):
        if nummern[i] == gesucht:
            return i
        else:
            return None

def summe(werte: list[int]) -> int:
    for wert in werte:
        ergebnis = 0
        ergebnis += wert
    return ergebnis

print(finden([101, 102, 203], 101), finden([101,
102, 203], 203))
print(summe([4, 5, 6]))
```

Ausgabe

```
0 None
6
```

```
def doppelt(x: int) -> int:
    print(x * 2)
    return x * 2

def halb(x: int) -> None:
    print(x // 2)

a = doppelt(3)
b = halb(9)
print(a, b, doppelt(a) + 1)
```

Ausgabe

```
6
4
12
6 None 13
```

Lösungswege (von links oben nach rechts unten): (1) `Manager` überschreibt `arbeiten` nicht, Tom bekommt die geerbte Version. (2) Ober- und Unterklasse schreiben in **dasselbe** Attribut `kontostand`, die 5000 sind weg; richtig wäre `self.laufzeit`. (3) `vorstellen` aus `Tier`, `laut` aus `Hund`; `Katze` setzt `laut` nicht um, bleibt abstrakt, der Fehler kommt beim **Erzeugen** `Katze()`, die erste Zeile ist da schon ausgegeben. (4) Laptop: beide Fähigkeiten, Router: nur Netzwerk, Monitor: nichts und kein Fehler, also 3 Zeilen. (5) `else: return None` beendet die Suche nach Runde 1; `ergebnis = 0` in der Schleife setzt jede Runde zurück, übrig bleibt 6. (6) `doppelt(3)` druckt 6 und liefert 6; `halb(9)` druckt 4 und liefert `None`; in der letzten Zeile wird zuerst `doppelt(a)` ausgewertet (druckt 12), dann gibt `print` die Zeile `6 None 13` aus.

Wird ergänzt

Nach den restlichen Terminen kommen dazu: Kaffeeshop Stufe 2/3 nach der Musterlösung, Standardbibliothek (VL08), Klausurvorbereitung, ein aktualisierter Abschnitt „Klausur auf einen Blick“ (Form, Hilfsmittel) und weitere Selbsttests.

Spick Spickzettel: Python-Syntax auf einer Seite

Werkzeuge

```
uv run datei.py          uv run ruff check .
uv run ruff format .     uv run ty check
uv add --dev pytest      uv run pytest
```

Imports, die du kennen musst

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from decimal import Decimal
from typing import Final, override
```

Typen und Operatoren (→ Ergebnis)

```
int 7 · float 3.5 · str "a" · bool True · None
7 / 2 → 3.5      7 // 2 → 3     7 % 2 → 1
-7 // 2 → -4     2 ** 3 → 8     "ab" * 2 → "abab"
int("42") → 42   str(3) → "3"   float("2.5") → 2.5
round(2.5) → 2   round(3.5) → 4 (zur geraden Zahl)
0.1 + 0.2 == 0.3 → False
Decimal("0.1") + Decimal("0.2") → 0.3   (Geld!)
x += 1  # x++ gibt es nicht
18 <= alter <= 65    not 0 → True
0 or "x" → "x"   3 and 0 → 0
a == b  Wert     a is None  Identität
MWST: Final = 0.19  # Konstante
```

Strings

```
s = "Kaffee"
s[0] → "K"   s[-1] → "e"   s[1:3] → "af"
len(s) → 6   s.lower() → "kaffee"
"aff" in s → True    "12".isdigit() → True
"a,b".split(",") → ["a", "b"]
f"{s} kostet {3.5:.2f} Euro" → Kaffee kostet 3.50 Euro
f"{s!r}" → 'Kaffee'   (mit Anführungszeichen)
```

Verzweigung

```
if x > 0:
    ...
elif x == 0:  # erster wahrer Zweig gewinnt
    ...
else:
    ...
match wahl:
    case "1" | "2":
        ...
    case "3" if summe > 25:  # Guard
        ...
    case _:  # sonst
        ...
text = "ja" if x else "nein"
```

Schleifen

```
range(5) → 0 1 2 3 4     range(1, 6) → 1 … 5
range(10, 0, -3) → 10 7 4 1   (Stopp nie dabei)
for ware in einkauf:  # jedes Element
for i, w in enumerate(liste): # Index + Element
while bedingung:  # Bedingung ändern!
break  # Schleife verlassen
continue  # nächste Runde
return  # ganze Funktion verlassen
```

Listen und Tupel

```
l = [3, 1, 2]
l.append(4) → [3, 1, 2, 4]
l.insert(0, 9)  l.remove(1)  x = l.pop()
len(l)  l[-1]  l[1:3]  sum(l)  min(l)  max(l)
sorted(l) → neue Liste    l.sort() → None!
b = a  # dieselbe Liste (2 Namen)
c = a.copy()  # neue Liste
a, b = b, a  # tauschen (Tupel)
t = (1, 2)  # unveränderbar
```

Funktionen

```
def brutto(netto: float, satz: float = 0.19) -> float:
    """Liefert den Bruttopreis."""
    return netto * (1 + satz)

brutto(100) → 119.0
brutto(100, satz=0.07)  # Schlüsselwort
def summe(*zahlen: int) -> int: ...
# ohne return → None   (print ≠ return)
def fakultaet(n: int) -> int:
    if n <= 1:  # Basisfall zuerst
        return 1
    return n * fakultaet(n - 1)

def main() -> None: ...
if __name__ == "__main__":
    main()
```

Alle Zeilen mit „→“ wurden in Python 3.13 nachgeprüft.

Klassen (VL5)

```
class Konto:
    anzahl = 0  # Klassenattribut

    def __init__(self, inhaber: str, stand: int = 0) -> None:
        self.inhaber = inhaber
        self._stand = stand  # "privat" (Konvention)
        Konto.anzahl += 1  # über die Klasse!

    @property
    def stand(self) -> int:  # lesen: k.stand
        return self._stand

    @stand.setter
    def stand(self, wert: int) -> None:
        if wert >= 0:  # Prüfregel
            self._stand = wert

    def __str__(self) -> str:  # return, nicht print
        return f"{self.inhaber}: {self._stand} Euro"

@dataclass
class Kunde:  # __init__, __repr__, == gratis
    name: str
    email: str
```

Vererbung, Abstraktion (VL6)

```
class Sparkonto(Konto):  # ist ein Konto
    def __init__(self, inhaber: str, zins: int) -> None:
        super().__init__(inhaber) # Pflicht!
        self.zins = zins

    @override
    def __str__(self) -> str:  # ergänzen
        return f"{super().__str__()} ({self.zins} %)"

    def __repr__(self) -> str:
        return f"{type(self).__name__}(inhaber=
{self.inhaber!r})"

class Form(ABC):  # ABC + abstractmethod
    @abstractmethod
    def flaeche(self) -> float: ...

class Rechnung(Dokument, Druckbar, Versendbar): ...
isinstance(s, Konto) → True   type(s) is Konto → False
for f in formen:  # Polymorphie
    gesamt += f.flaeche()
```

Schleifenmuster

```
summe = 0  # vor der Schleife
for p in preise:
    summe += p  # in der Schleife
return summe  # nach der Schleife

for p in produkte:  # finden
    if p.nummer == nummer:
        return p
return None  # NACH der Schleife

treffer = []  # sammeln → [] statt None
for p in produkte:
    if text.lower() in p.name.lower():
        treffer.append(p)
```

Fehler in einem Satz

`SyntaxError`	`:` oder Klammer fehlt
`IndentationError`	Block nicht eingerückt
`NameError`	Name unbekannt, Import fehlt
`TypeError`	falscher Typ, Argument fehlt, abstrakt erzeugt, `None + 1`
`ValueError`	`int("zwei")`
`IndexError`	Index zu groß
`AttributeError`	Attribut fehlt, `super().__init__` vergessen, `None.name`
`RecursionError`	Basisfall nie erreicht